

B. Tech. (Computer Science and Engineering)

A22 Regulation

MACHINE LEARNING

LAB MANUAL

Subject Code: Code: 9EC65

Regulations: A22 JNTUH

Class: III Year B.Tech .II Semester

MACHINE LEARNING LAB

Course Objective: The objective of this lab is to get an overview of the various machine learning techniques and can able to demonstrate them using python.

Course Outcomes: After the completion of the course the student can able to:

1. Generalize complexity of Machine Learning algorithms in real-world data and their limitations.[L2]

Program Outcomes

	Program Outcomes														
Cou rse outc ome s	PO 1	PO 2	PO 3	PO4	PO5	PO6	PO7	PO8	PO9	PO 10	PO1 1	PO 12	PS O1	PS O2	PS O3
CO1	3	3	3	3	3	2	3	--	---	--	--	3	3	3	2
Ave rage	3	3	3	3	3	2	3	--	--	--	--	3	3	3	2

PO's	1	2	3	4	5	6	7	8	9	10	11	12
Level	H	H	H	H	H	M	H	--	--	---	--	H

B. Tech. (INFORMATION TECHNOLOGY)

A22 Regulation

(Applicable for the Batches admitted from 2022 & 2023)



DEPARTMENT OF INFORMATION TECHNOLOGY

SREENIDHI INSTITUTE OF SCIENCE AND TECHNOLOGY

(An Autonomous Institution approved by UGC and affiliated to JNTUH)
Yamnampet, Ghatkesar, Hyderabad - 501 301

2024-2025

DEPARTMENT OF INFORMATION TECHNOLOGY

MACHINE LEARNING LAB

LIST OF EXPERIMENTS

S.No.	Name of the Experiment																														
1.	The probability that it is Friday and that a student is absent is 3 %. Since there are 5 school days in a week, the probability that it is Friday is 20%. What is the probability that a student is absent given that today is Friday? Apply Baye's rule in python to get the result. (Ans: 15%)																														
2.	Extract the data from database using python																														
3.	Implement k-nearest neighbours classification using python																														
4.	Given the following data, which specify classifications for nine combinations of VAR1 and VAR2 predict a classification for a case where VAR1=0.906 and VAR2=0.606, using the result of k-means clustering with 3 means (i.e., 3 centroids) <table><tr><th>VAR1</th><th>VAR2</th><th>CLASS</th></tr><tr><td>1.713</td><td>1.586</td><td>0</td></tr><tr><td>0.180</td><td>1.786</td><td>1</td></tr><tr><td>0.353</td><td>1.240</td><td>1</td></tr><tr><td>0.940</td><td>1.566</td><td>0</td></tr><tr><td>1.486</td><td>0.759</td><td>1</td></tr><tr><td>1.266</td><td>1.106</td><td>0</td></tr><tr><td>1.540</td><td>0.419</td><td>1</td></tr><tr><td>0.459</td><td>1.799</td><td>1</td></tr><tr><td>0.773</td><td>0.186</td><td>1</td></tr></table>	VAR1	VAR2	CLASS	1.713	1.586	0	0.180	1.786	1	0.353	1.240	1	0.940	1.566	0	1.486	0.759	1	1.266	1.106	0	1.540	0.419	1	0.459	1.799	1	0.773	0.186	1
VAR1	VAR2	CLASS																													
1.713	1.586	0																													
0.180	1.786	1																													
0.353	1.240	1																													
0.940	1.566	0																													
1.486	0.759	1																													
1.266	1.106	0																													
1.540	0.419	1																													
0.459	1.799	1																													
0.773	0.186	1																													
5.	The following training examples map descriptions of individuals onto high, medium and low																														

	credit-worthiness. mediumskiingdesign single twenties no - >highRiskhigh golf tradingmarriedfortiesyes ->lowRiskow speedway transport married thirties yes - >medRisk mediumfootball banking single thirties yes ->lowRiskhigh flyingmedia marriedfiftiesyes ->highRiskow football security single twenties no ->medRisk mediumgolf media single thirties yes ->medRisk mediumgolf transport married forties yes - >lowRiskhigh skiingbanking singlethirties yes ->highRiskow golf unemployedmarriedfortiesyes->highRisk Inputattributesare(fromlefttoright)income,recreation, job,status,age- group, home-owner. Find the unconditional probability of 'golf' and the conditional probability of 'single' given 'medRisk' in the dataset?
6.	Implementlinearregressionusingpython.
7.	ImplementNaïveBayestheoremtoclassifytheEnglishtext
8.	Implementanalgorithmto demonstratethesignificanceofgenetic algorithm
9.	Implement the finite words classification system using Back-propagation algorithm.
10.	Implement Classification on a sample data set using SVM?

DEPARTMENT OF INFORMATION TECHNOLOGY MACHINE LEARNING LAB

INTRODUCTION to LAB:

Machine Learning is used anywhere from automating mundane tasks to offering intelligent insights, industries in every sector try to benefit from it. You may already be using a device that utilizes it. For example, a wearable fitness tracker like Fitbit, or an intelligent home assistant like Google Home. But there are much more examples of ML in use.

- **Prediction:** Machine learning can also be used in the prediction systems. Considering the loan example, to compute the probability of a fault, the system will need to classify the available data in groups.
- **Image recognition:** Machine learning can be used for face detection in an image as well. There is a separate category for each person in a database of several people.
- **Speech Recognition:** It is the translation of spoken words into the text. It is used in voice searches and more. Voice user interfaces include voice dialing, call routing, and appliance control. It can also be used as a simple data entry and the preparation of structured documents.
- **Medical diagnoses:** ML is trained to recognize cancerous tissues.
- **Financial industry:** and trading: companies use ML in fraud investigations and credit checks.

Types of Machine Learning?

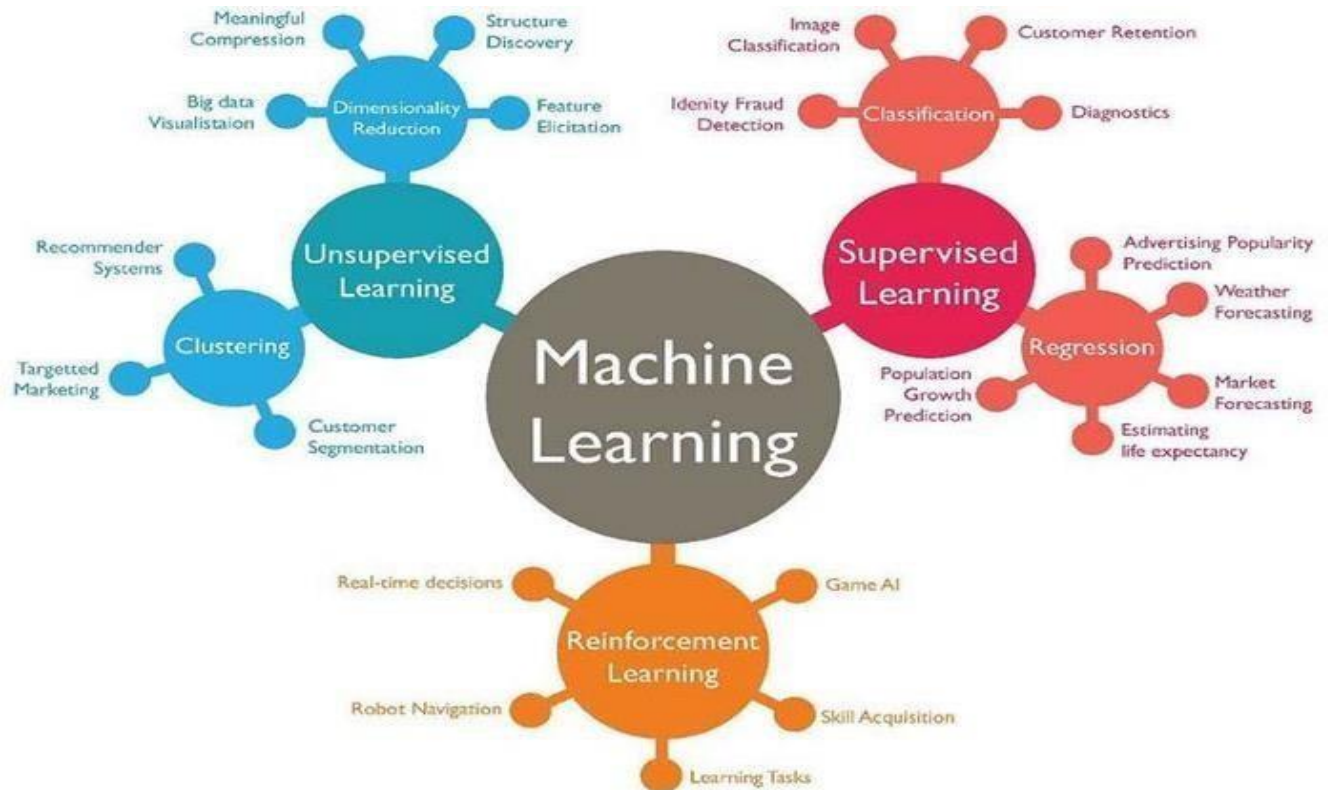
Machine learning can be classified into 3 types of algorithms

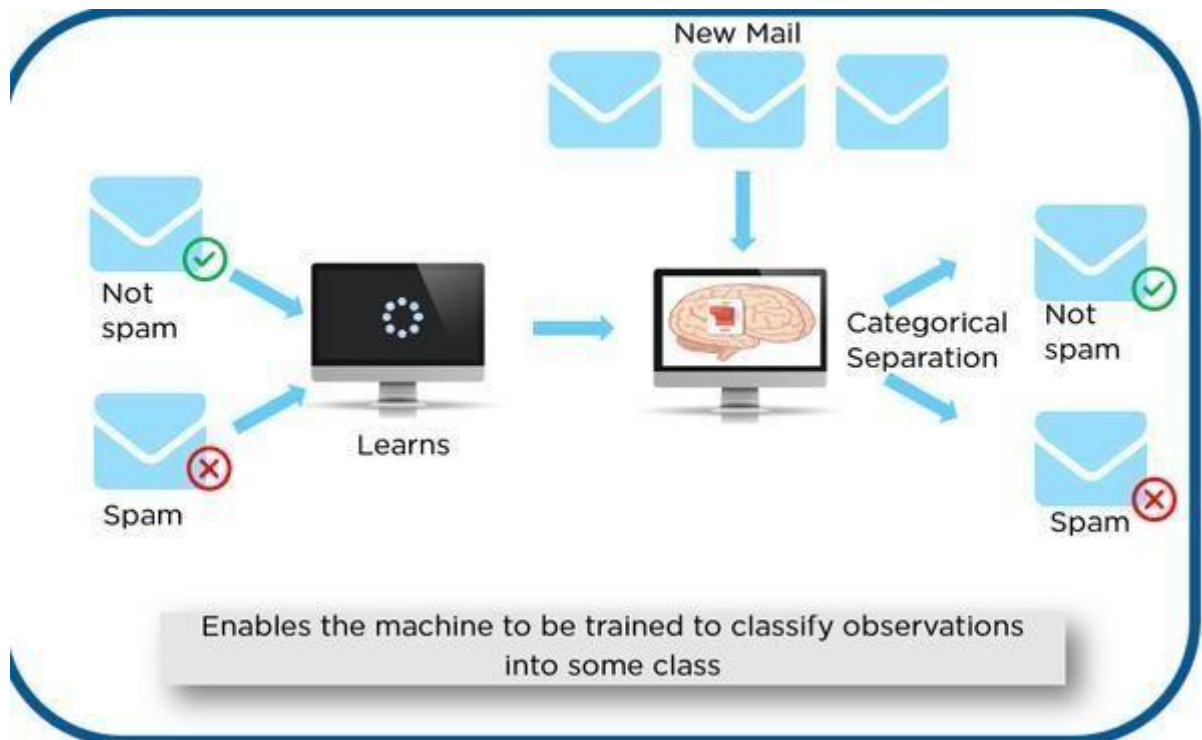
1. Supervised Learning
2. Unsupervised Learning
3. Reinforcement Learning

Overview of Supervised Learning Algorithm

In Supervised learning, an AI system is presented with data which is labeled, which means that each data is tagged with the correct label.

The goal is to approximate the mapping function so well that when you have new input data (x) that you can predict the output variables (Y) for that data.





As shown in the above example, we have initially taken some data and marked them as ‘Spam’ or ‘Not Spam’. This labeled data is used by the training supervised model, this data is used to train the model.

Once it is trained we can test our model by testing it with some test new mails and checking if the model is able to predict the right output.

Types of Supervised learning

- **Classification:** A classification problem is when the output variable is a category, such as “red” or “blue” or “disease” and “no disease”.
- **Regression:** A regression problem is when the output variable is a real value, such as “dollars” or “weight”.

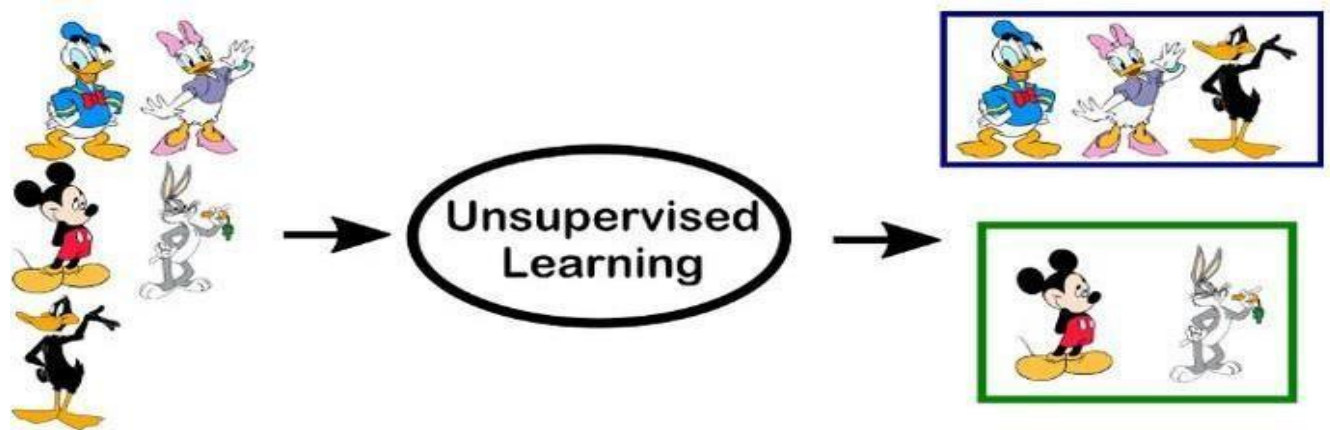
Overview of Unsupervised Learning Algorithm

In unsupervised learning, an AI system is presented with unlabeled, uncategorized data and the system’s algorithms act on the data without prior training. The output is dependent upon the coded algorithms. Subjecting a system to unsupervised learning is one way of testing AI.

Types of Unsupervised learning:

- **Clustering:** A clustering problem is where you want to discover the inherent groupings in the data, such as grouping customers by purchasing behavior.

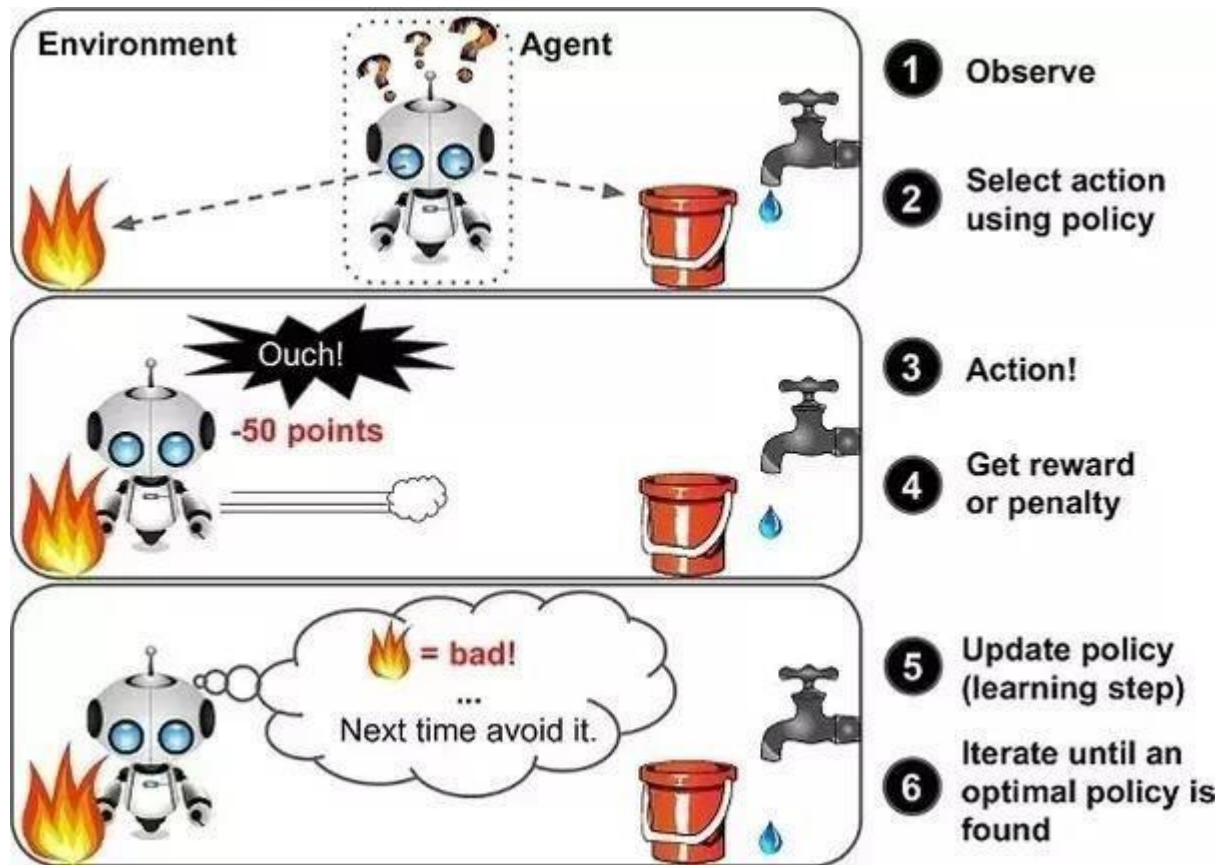
- **Association:** An association rule learning problem is where you want to discover rules that describe large portions of your data, such as people that buy X also tend to buy Y.



Example of Unsupervised Learning

Overview of Reinforcement Learning

A reinforcement learning algorithm, or agent, learns by interacting with its environment. The agent receives rewards by performing correctly and penalties for performing incorrectly. The agent learns without intervention from a human by maximizing its reward and minimizing its penalty. It is a type of dynamic programming that trains algorithms using a system of reward and punishment.



in the above example, we can see that the agent is given 2 options i.e. a path with water or a path with fire. A reinforcement algorithm works on reward a system i.e. if the agent uses the fire path then the rewards are subtracted and agent tries to learn that it should avoid the fire path. If it had chosen the water path or the safe path then some points would have been added to the reward points, the agent then would try to learn what path is safe and what path isn't.

It is basically leveraging the rewards obtained; the agent improves its environment knowledge to select the next action.

PROGRAM1(a)

Write a python program to import and export the data using pandas library

1. Manual Function

Go to Google Page and find www.kaggle.com. Select Datasets and find Titanic dataset, then download train.csv file and save it to desktop.

1. Read a CSV file

```
import pandas as pd url='C:/Users/SNIST/Desktop/train.csv'
dataframe=pd.read_csv(url)dataframe.head(5)
```

2. Write a CSVfile

```
Import pandas as pd
marks_data=pd.DataFrame({'ID':{0:23,1:43,2:12,3:13,4:67,5:89},'NAME':{0:'Ram',1:'Deep',2:'Yash',3:'Arjun',4:'Aditya',5:'Divya'},'Marks':{0:89,1:92,2:45,3:78,4:56,5:76},'Grade':{0:'b',1:'a',2:'f',3:'c',4:'e',5:'c'}})

filename='C:/Users/SNIST/Desktop/Marksdata.xlsx'm
arks_data.to_excel(filename)

print('Data frame written to Excel')
```

3. Read an Excel File import pandas as

```
pdurl='C:/Users/SNIST/Desktop/train.csv.xls'dataframe=p
d.read_excel(url)dataframe.head(5)
```

4. Write an Excel file

```
importpandas as pd
marks_data=pd.DataFrame({'ID':{0:23,1:43,2:12,3:13,4:67,5:89},'NAME':{0:'Ram',1:'Deep',2:'Yash',3:'Arjun',4:'Aditya',5:'Divya'},'Marks':{0:89,1:92,2:45,3:78,4:56,5:76},'Grade':{0:'b',1:'a',2:'f',3:'c',4:'e',5:'c'}})
filename='C:/Users/SNIST/Desktop/Marksdata.csv'marks_data.to_csv(filename)

print('Data frame written to CSV');
```

Output:

PROGRAM1(b)

The probability that it is Friday and that a student is absent is 3%. Since there are 5 school days in a week, the probability that it is Friday is 20%. What is the probability that a student is absent given that today is Friday? Apply Bayes' rule in python to get the result. (Ans: 15%)

AIM: To find the probability that a student is absent given that today is Friday.

DESCRIPTION:

Machine learning is a method of data analysis that automates analytical model building of data set. Using the implemented algorithms that iteratively learn from data, machine learning allows computers to find hidden insights without being explicitly programmed where to look. Naive Bayes algorithm is one of the most popular machine learning technique. In this article we will look how to implement Naive Bayes algorithm using python.

Before someone can understand Bayes' theorem, they need to know a couple of related concepts first, namely, the idea of Conditional Probability, and Bayes' Rule.

Conditional Probability is just what is the probability that something will happen, given that something else has already happened.

Let say we have a collection of people. Some of them are singers. They are either male or female. If we select a random sample, what is the probability that this person is a male? what is the probability that this person is a male and singer? Conditional Probability is the best option here. We can calculate probability like,

$$P(\text{Singer \& Male}) = P(\text{Male}) \times P(\text{Singer} / \text{Male})$$

What is Bayes rule ?

We can simply define Bayes rule like this. Let $A_1, A_2, \dots, A_n$ be a set of mutually exclusive events that together form the sample space S . Let B be any event from the same sample space, such that $P(B) > 0$. Then, $P(A_k | B) = P(A_k \cap B) / P(A_1 \cap B) + P(A_2 \cap B) + \dots + P(A_n \cap B)$

What is Bayes classifier?

Naive Bayes classifiers are a family of simple probabilistic classifiers based on applying Bayes' theorem with strong (naive) independence assumptions between the features in machine learning. Basically we can use above theories and equations for classification problem.

SOURCE CODE:

- . To find the probability that a student is absent given that today is Friday, we use the formula for conditional probability:

$$P(\text{Absent} \mid \text{Friday}) = P(\text{Absent and Friday}) / P(\text{Friday})$$

From the problem:

$$P(\text{Absent and Friday}) = 0.03 \text{ (3\%)}$$

$$P(\text{Friday}) = 0.20 \text{ (20\%)}$$

$$P(\text{Absent} \mid \text{Friday}) = 0.03 / 0.20 = 0.15$$

So, the probability that a student is absent given that today is Friday is **15%**.

The Python code that implements Bayes' Rule to calculate the conditional probability of a student being absent given that today is Friday.

```
def bayes_rule(p_absent_and_friday, p_friday):  
    return p_absent_and_friday / p_friday  
  
# Given probabilities  
p_absent_and_friday = 0.03 # P(Absent and Friday)  
p_friday = 0.20           # P(Friday)  
  
# Calculate P(Absent | Friday)  
p_absent_given_friday = bayes_rule(p_absent_and_friday, p_friday)  
  
print("The probability that a student is absent given that today is Friday is:",  
      p_absent_given_friday*100,"%")
```

Output: The probability that a student is absent given that today is Friday is: 15.0 %

Output:15

```
: probAbsentFriday=0.03
  probFriday=0.2
  # bayes Formula
  #p(Absent|Friday)=p(Friday|Absent)p(Absent)/p(Friday)
  #p(Friday|Absent)=p(FridaynAbsent)/p(Absent)
  # Therefore the result is:
  bayesResult=(probAbsentFriday/probFriday)
  print(bayesResult * 100)
```

15.0

1.5.VIVAQUESTIONS&ANSWERS

1. What are Bayesian Networks(BN)?

Bayesian Network is used to represent the graphical model for probability relationship among a set of variables. Bayes' theorem is a way to figure out conditional probability. Conditional probability is the probability of an event happening, given that it has some relationship to one or more other events. For example, your probability of getting a parking space is connected to the time of day you park, where you park, and what conventions are going on at anytime. Bayes' theorem is slightly more nuanced. In a nutshell, it gives you the actual probability of an **event** given information about **tests**.

- “Events” Are different from “tests.” For example, there is a **test** for liver disease, but that’s separate from the **event** of actually having liver disease.
- **Tests are flawed:** just because you have a positive test does not mean you actually have the disease. Many tests have a high false positive rate. **Rare events tend to have higher false positive rates** than more common events. We’re not just talking about medical tests here. For example, spam filtering can have high false positive rates. Bayes’ theorem takes the test results and calculates your *real probability* that the test has identified the event.

2. Can you give any real time example using Bayes’ Theorem (liver disease).

You might be interested in finding out a patient’s probability of having liver disease if they are an alcoholic. “Being an alcoholic” is the **test** (kind of like a litmus test) for liver disease.

- **A** could mean the event “Patient has liver disease.” Past data tells you that 10% of patients entering your clinic have liver disease. $P(A) = 0.10$.
- **B** could mean the litmus test that “Patient is an alcoholic.” Five percent of the clinic’s patients are alcoholics. $P(B) = 0.05$.
- You might also know that among those patients diagnosed with liver disease, 7% are alcoholics. This is your **B|A**: the probability that a patient is alcoholic, given that they have liver disease, is 7%.

Bayes’ theorem tells you:

$$P(A|B) = (0.07 * 0.1) / 0.05 = 0.14$$

In other words, if the patient is an alcoholic, their chances of having liver disease is 0.14 (14%).

This is a large increase from the 10% suggested by past data. But it’s still unlikely that any particular patient has liver disease.

3. Bayes’ Theorem Examples 2: what is the probability that they will be prescribed pain pills?

Another way to look at the theorem is to say that one event follows another. Above I said “tests” and “events”, but it’s also legitimate to think of it as the “first event” that leads to the “second event.” There’s no one right way to do this: use the terminology that makes most sense to you.

In a particular pain clinic, 10% of patients are prescribed narcotic pain killers. Overall, five percent of the clinic’s patients are addicted to narcotics (including pain killers and illegal substances). Out of all the people prescribed pain pills, 8% are addicts. *If a patient is an addict, what is the probability that they will be prescribed pain pills?*

Step 1: Figure out what you're event "A" is from the question. That information is in the italicized part of this particular question. The event that happens first (A) is being prescribed pain pills. That's given as 10%.

Step 2: Figure out what you're event "B" is from the question. That information is also in the italicized part of this particular question. Event B is being an addict. That's given as 5%.

Step 3: Figure out what the probability of event B (Step 2) given event A (Step 1). In other words, find what $P(B|A)$ is. We want to know "Given that people are prescribed pain pills, what's the probability they are an addict?" That is given in the question as 8%, or .8.

Step 4: Insert your answers from Steps 1, 2 and 3 into the formula and solve. $P(A|B) = P(B|A) * P(A) / P(B)$
 $= (0.08 * 0.1) / 0.05 = 0.16$

The probability of an addict being prescribed pain pills is 0.16 (16%).

4. *Bayes' Theorem Examples 3: the Medical Test if a person gets a positive test result.*

what are the odds they actually have the genetic defect?

A slightly more complicated example involves a medical test (in this case, a genetic test):

There are **several forms of Bayes' Theorem** out there, and they are all equivalent (they are just written in slightly different ways). In this next equation, "X" is used in place of "B." In addition, you'll see some changes in the denominator. The proof of why we can rearrange the equation like this is beyond the scope of this article (otherwise it would be 5,000 words instead of 2,000!). However, if you come across a question involving medical tests, you'll likely be using this alternative formula to find the answer:

$$Pr(A|X) = \frac{Pr(X|A) Pr(A)}{Pr(X|A) Pr(A) + Pr(X|\sim A) Pr(\sim A)}$$

1% of people have a certain genetic defect.

90% of tests for the gene detect the defect (true positives). 9.6%

of the tests are false positives.

If a person gets a positive test result, **what are the odds they actually have the genetic defect?**

The first step in solving Bayes' theorem problems is to assign letters to events:

- A = chance of having the faulty gene. That was given in the question as 1%. That also means the probability of *not* having the gene ($\sim A$) is 99%.
- X = A positive test result.

Sol:

1. $P(A|X)$ = Probability of having the gene given a positive test result.
2. $P(X|A)$ = Chance of a positive test result given that the person actually has the gene. That was given in the question as 90%.
3. $p(X|\sim A)$ = Chance of a positive test if the person *doesn't* have the gene. That was given in the question as 9.6%

Now we have all of the information we need to put into the equation:

$$P(A|X) = (.9 * .01) / (.9 * .01 + .096 * .99) = 0.0865 (8.65\%).$$

The probability of having the faulty gene on the test is 8.65%.

5. Given the following statistics, what is the probability that a woman has cancer if she has a positive mammogram result?

- One percent of women over 50 have breast cancer.
- Ninety percent of women who have breast cancer test positive on mammograms.
- Eight percent of women will have false positives.

Step 1: Assign events to A or X . You want to know what a woman's probability of having cancer is, given a positive mammogram. For this problem, actually having cancer is A and a positive test result is X .

Step 2: List out the parts of the equation (this makes it easier to work the actual equation): $P(A) = 0.01$

$$P(\sim A) = 0.99$$

$$P(X|A) = 0.9$$

$$P(X|\sim A) = 0.08$$

Step 3: Insert the parts into the equation and solve. Note that as this is a medical test, we're using the form of the equation from example #2:

$$(0.9 * 0.01) / ((0.9 * 0.01) + (0.08 * 0.99)) = 0.10.$$

The probability of a woman having cancer, given a positive test result, is 10%.

PROGRAM2

2. EXTRACT THE DATA FROM DATABASE USING PYTHON

You'll learn the following MySQL SELECT operations from Python using a 'MySQLConnector Python' module.

- ExecutetheSELECTqueryandprocesstheresult setreturned bythequeryinPython.
- UsePython**variables in a where clause** of aSELECTquery to passdynamic values.
- Usefetchall(),fetchmany(),andfetchone()methodsof a cursor classtofetchallor limited rows from atable.
- PythonSelectfromMySQLTable

This article demonstrates how to **select rows of a MySQL table in Python**.

You'll learn the following MySQL SELECT operations from Python using a 'MySQL Connector Python' module

- ExecutetheSELECTqueryandprocesstheresult setreturnedbythequeryinPython.
- Use Python **variables in a where clause** of aSELECTquery to passdynamicvalues.
- Use fetchall(),fetchmany(), and fetchone() methods of a cursor class to fetchallor limited rows fromatable.
- StepstofetchrowsfromaMySQLdatabasetableFollow thesesteps:–

SOURCE CODE:

Step 1: Create a Table (students) in Mysql Database (SampleDB)

Open Command prompt and then execute the following command to enter into MySQL prompt.

```
mysql -u root -p
create database SampleDB;
use SampleDB;
CREATE TABLE students (sid VARCHAR(10),sname VARCHAR(10),age int);
INSERT INTO students VALUES('22311A0521','AAA',23);
INSERT INTO students VALUES('22311A0522','BBB',22);
INSERT INTO students VALUES('22311A0523','CCC',25);
```

```
INSERT INTO students VALUES('22311A0524','DDD',24);
```

```
pip install mysql.connector
```

```
# to connect with mysql database through python
```

```
import mysql.connector
```

```
# Create the connection object
```

```
myconn = mysql.connector.connect(host = "localhost", user = "root",passwd = "",database="SampleDB")
```

```
# Creating the cursor object
```

```
cur = myconn.cursor()
```

```
# Executing the query
```

```
cur.execute("select * from students")
```

```
# Fetching the rows from the cursor object
```

```
result = cur.fetchall()
```

```
print("Student Details are :")
```

```
# Printing the result
```

```
for x in result:
```

```
    print(x);
```

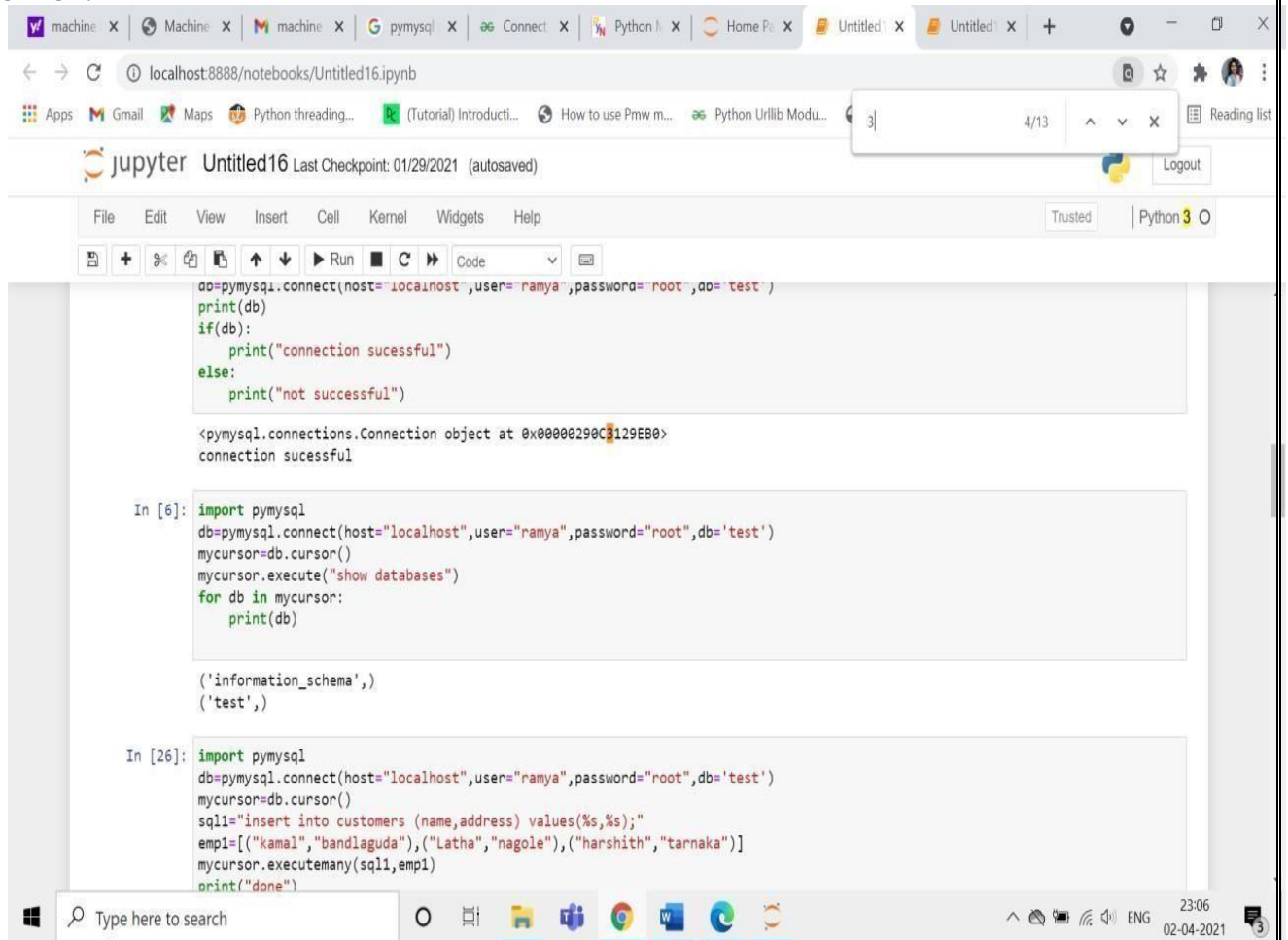
```
# Commit the transaction
```

```
myconn.commit()
```

```
# Close the connection
```

```
myconn.close()
```

OUTPUT:



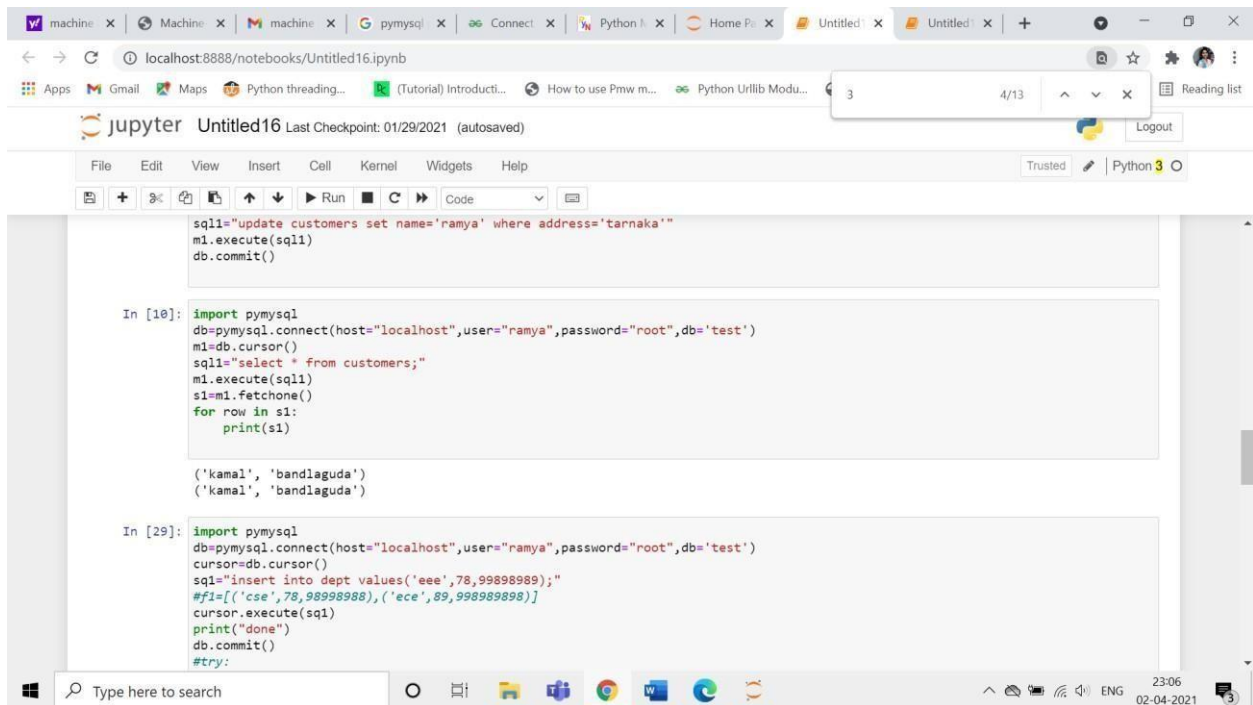
```
db=pymysql.connect(host='localhost',user='ramya',password='root',db='test')
print(db)
if(db):
    print("connection sucessful")
else:
    print("not successful")

<pymysql.connections.Connection object at 0x00000290C3129EB0>
connection sucessful

In [6]: import pymysql
db=pymysql.connect(host="localhost",user="ramya",password="root",db='test')
mycursor=db.cursor()
mycursor.execute("show databases")
for db in mycursor:
    print(db)

('information_schema',)
('test',)

In [26]: import pymysql
db=pymysql.connect(host="localhost",user="ramya",password="root",db='test')
mycursor=db.cursor()
sql1="insert into customers (name,address) values(%,%s);"
empl=[("kamal","bandlaguda"),("Latha","nagole"),("harshith","tarnaka")]
mycursor.executemany(sql1,empl)
print("done")
```



```
sql1="update customers set name='ramya' where address='tarnaka'"
m1.execute(sql1)
db.commit()

In [10]: import pymysql
db=pymysql.connect(host="localhost",user="ramya",password="root",db='test')
m1=db.cursor()
sql1="select * from customers;"
m1.execute(sql1)
s1=m1.fetchone()
for row in s1:
    print(s1)

('kamal', 'bandlaguda')
('kamal', 'bandlaguda')

In [29]: import pymysql
db=pymysql.connect(host="localhost",user="ramya",password="root",db='test')
cursor=db.cursor()
sql="insert into dept values('eee',78,99898989);"
#f1=[('cse',78,9898988),('ece',89,99898988)]
cursor.execute(sql)
print("done")
db.commit()
#try:
```

VIVAQUESTIONSANDANSWERS

1. How to select from a MySQL table using Python?

Connect to MySQL from Python

Refer to [Python MySQL database connection](#) to connect to MySQL database from Python using MySQL Connector module

Define a SQL SELECT Query

Next, prepare a SQL SELECT query to fetch rows from a table. You can select all or limited rows based on your requirement. If the where condition is used, then it decides the number of rows to fetch. For example, `SELECT col1, col2, ..., coln FROM MySQL_table WHERE id = 10;` This will return row number 10.

Get Cursor Object from Connection

Next, use a `connection.cursor()` method to create a cursor object. This method creates a new MySQL Cursor object.

Execute the SELECT query using execute() method

Execute the select query using the cursor.execute() method.

Extract all rows from a result

After successfully executing a Select operation, Use the fetchall() method of a cursor object to get all rows from a query result. it returns a list of rows.

Iterate each row

Iterate a row list using a for loop and access each row individually (Access each row's column data using a column name or index number.)

Close the cursor object and database connection object

use cursor.close() and connection.close() method to close open connections after your work completes.

2. What are the methods to fetch data returned by a cursor.execute()

- cursor.fetchall() to fetch all rows
- cursor.fetchone() to fetch a single row
- cursor.fetchmany(SIZE) to fetch limited rows

PROGRAM3

3. IMPLEMENT K-NEAREST NEIGHBORS CLASSIFICATION USING PYTHON

AIM: To Implement k-neighbours classification using python

DESCRIPTION:

This algorithm is used to solve the classification model problems. K-nearest neighbor or K-NN algorithm basically creates an imaginary boundary to classify the data. When new data points come in, the algorithm will try to predict that to the nearest of the boundary line.

Therefore, larger k value means smoother curves of separation resulting in less complex models. Whereas, smaller k value tends to over fit the data and resulting in complex models.

It's very important to have the right k-value when analyzing the dataset to avoid over fitting and under fitting of the dataset.

KNN MODEL REPRESENTATION:

The model representation for KNN is the entire training dataset. It is as simple as that.

KNN has no model other than storing the entire dataset, so there is no learning required.

Efficient implementations can store the data using complex data structures like k-d tree to make look-up and matching of new patterns during prediction efficient.

Because the entire training dataset is stored, you may want to think carefully about the consistency of your training data. It might be a good idea to curate it, update it often as new data becomes available and remove erroneous and outlier data.

- The **k-nearest neighbor algorithm** is imported from the **scikit-learn** package.
- Create feature and target variables.
- Split data into training and test data.
- Generate a **k-NN** model using **neighbors** value.
- Train or fit the data into the model.
- Predict the future.

SOURCE CODE:

```
pip install scikit-learn
```

Considering Iris Dataset (70% training set and 30% test dataset)

The features of iris dataset sepal length (cm), sepal width (cm), petal length (cm), petal width (cm)

```
# Import necessary modules
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.datasets import load_iris
```

```
import random
```

```
# Loading data
```

```
data_iris = load_iris()
```

```
# To get list of target names
```

```
label_target = data_iris.target_names
```

```
print("Sample Data from Iris Dataset")
```

```
print("""*30)
```

```
# to display the sample data from the iris dataset
```

```
for i in range(10):
```

```
    rn = random.randint(0,120)
```

```
    print(data_iris.data[rn],"==>",label_target[data_iris.target[rn]])
```

```
# Create feature and target arrays
```

```
X = data_iris.data
```

```
y = data_iris.target
```

```
# Split into training and test set
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size = 0.3, random_state=1)
```

```
print("The Training dataset length: ",len(X_train))
```

```
print("The Testing dataset length: ",len(X_test))
```

```
try:
```

```

nn = int(input("Enter number of neighbors :"))
knn = KNeighborsClassifier(nn)

knn.fit(X_train, y_train)
# to display the score
print("The Score is :",knn.score(X_test, y_test))
# To get test data from the user
test_data = input("Enter Test Data :").split(",")
for i in range(len(test_data)):
    test_data[i] = float(test_data[i])

print()
v = knn.predict([test_data])
print("Predicted output is :",label_target[v])
except:
    print("Please supply valid input.....")

```

Output:

```

from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
import random

data_iris = load_iris()

label_target = data_iris.target_names
print("Sample Data from Iris Dataset")
print("***30")
for i in range(10):
    rn = random.randint(0,120)
    print(data_iris.data[rn],"==>",label_target[data_iris.target[rn]])

```

```

Sample Data from Iris Dataset
*****
[5.1 2.5 3.  1.1] ==> versicolor
[6.5 3.2 5.1 2. ] ==> virginica
[4.8 3.1 1.6 0.2] ==> setosa
[6.1 2.8 4.  1.3] ==> versicolor
[5.2 2.7 3.9 1.4] ==> versicolor
[7.  3.2 4.7 1.4] ==> versicolor
[5.9 3.  4.2 1.5] ==> versicolor
[5.1 3.4 1.5 0.2] ==> setosa
[7.7 2.6 6.9 2.3] ==> virginica
[4.6 3.6 1.  0.2] ==> setosa

```

```

X = data_iris.data
y = data_iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3, random_state=1)
print("The Training dataset length: ",len(X_train))
print("The Testing dataset length: ",len(X_test))

```

```

The Training dataset length: 105
The Testing dataset length: 45

```

```

nn = int(input("Enter number of neighbors :"))
knn = KNeighborsClassifier(nn)

knn.fit(X_train, y_train)
# to display the score
print("The Score is :",knn.score(X_test, y_test))

```

```

Enter number of neighbors :3
The Score is : 0.9777777777777777

```

```

# To get test data from the user
test_data = input("Enter Test Data :").split(",")
for i in range(len(test_data)):
    test_data[i] = float(test_data[i])
print()

v = knn.predict([test_data])
print("Predicted output is :",label_target[v])

```

```

Enter Test Data :4.5,6.7,3.5,6
Predicted output is : ['virginica']

```

```

Sample Data from Iris Dataset
*****
[6.1 2.8 4.7 1.2] ==> versicolor
[4.9 3.1 1.5 0.2] ==> setosa
[5.4 3.9 1.3 0.4] ==> setosa
[4.9 2.4 3.3 1. ] ==> versicolor
[6.4 3.2 5.3 2.3] ==> virginica
[6.3 3.3 6.  2.5] ==> virginica
[6.5 3.  5.8 2.2] ==> virginica
[5.6 2.7 4.2 1.3] ==> versicolor
[5.1 2.5 3.  1.1] ==> versicolor
[4.4 3.  1.3 0.2] ==> setosa
The Training dataset length: 105
The Testing dataset length: 45
Enter number of neighbors :10
The Score is : 0.9777777777777777
Enter Test Data :5.0,3.3,1.4,0.3

Predicted output is : ['setosa']

```

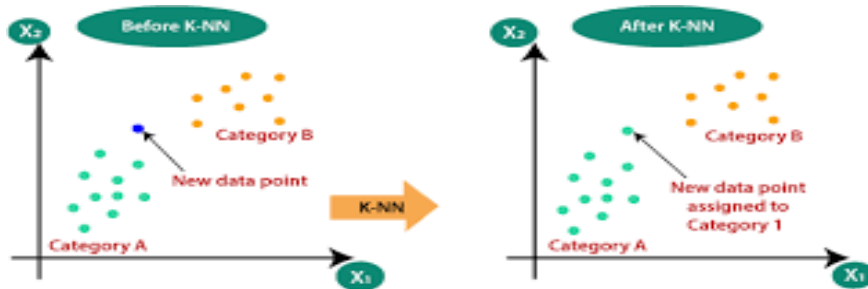
VAQUESTIONS&ANSWERS

1. WhatistheKNNAlgorithm?

KNN(K-nearestneighbors) is a **supervised** learning and **non-parametric** algorithm that can be used to solve both classification and regression problem statements.

It uses data in which there is a target column present **i.e, labeled data** to model a function to produce an output for the unseen data. It uses the Euclidean distance formula to compute the distance between the data points for classification or prediction.

The main objective of this algorithm is that similar data points must be close to each other so it uses the distance to calculate the similar points that are close to each other.



2. Why is KNN a non-parametric Algorithm?

The term “**non-parametric**” refers to not making any assumptions on the underlying data distribution. These methods do not have any fixed numbers of parameters in the model.

Similarly in KNN, the model parameters grow with the training data by considering each training case as a parameter of the model. So, KNN is a non-parametric algorithm.

3. What is “K” in the KNN Algorithm?

K represents the number of nearest neighbors you want to select to predict the class of a given item, which is coming as an unseen dataset for the model.

4. Why is the odd value of “K” preferred over even values in the KNN Algorithm?

The odd value of K should be preferred over even values in order to ensure that there are no ties in the voting. If the square root of a number of data points is even, then add or subtract 1 to it to make it odd.

5. How does the KNN algorithm make the predictions on the unseen dataset?

The following operations have happened during each iteration of the algorithm. For each of the unseen or test data point, the kNN classifier must:

Step-1: Calculate the distances of test point to all points in the training set and store them

Step-2: Sort the calculated distances in increasing order

Step-3: Store the K nearest points from our training dataset

Step-4: Calculate the proportions of each class

Step-5: Assign the class with the highest proportion

PROGRAM-4

4. Given the following data, which specify classifications for nine combinations of VAR1 and VAR2, predict a classification for a case where VAR1=0.906 and VAR2=0.606, using the result of k-means clustering with 3 means (i.e., 3 centroids)

VAR1	VAR2	CLASS
1.713	1.586	0
0.180	1.786	1
0.353	1.240	1
0.940	1.566	0
1.486	0.759	1
1.266	1.106	0
1.540	0.419	1
0.459	1.799	1
0.773	0.186	1

SOURCE CODE:

Sol. We need apply k-means clustering with 3 means (i.e., 3 centroids) and predict the class for the VAR1=0.906 and VAR2=0.606

```
pip install scikit-learn
from sklearn.cluster import KMeans
import numpy as np
X = np.array([[1.713,1.586], [0.180,1.786], [0.353,1.240],
[0.940,1.566], [1.486,0.759], [1.266,1.106],[1.540,0.419],[0.459,1.799],[0.773,0.186]])
y=np.array([0,1,1,0,1,0,1,1,1])
kmeans = KMeans(n_clusters=3, random_state=0).fit(X,y)
print("The input data is ")
print("VAR1 \t VAR2 \t CLASS")
i=0
for val in X:
    print(val[0],"\t",val[1],"\t",y[i])
    i+=1
print("\n"*20)
# To get test data from the user
print("The Test data to predict ")
test_data = []
VAR1 = float(input("Enter Value for VAR1 :"))
```

```

VAR2 = float(input("Enter Value for VAR2 :"))
test_data.append(VAR1)
test_data.append(VAR2)
print("="*20)
print("The predicted Class is : ",kmeans.predict([test_data]))

```

Output:

```

from sklearn.cluster import KMeans
import numpy as np

```

```

X = np.array([[1.713,1.586], [0.180,1.786], [0.353,1.240],
[0.940,1.566], [1.486,0.759], [1.266,1.106],[1.540,0.419],[0.459,1.799],[0.773,0.186]])
y=np.array([0,1,1,0,1,0,1,1,1])
kmeans = KMeans(n_clusters=3, random_state=0).fit(X,y)
print("The input data is ")
print("VAR1 \t VAR2 \t CLASS")
i=0
for val in X:
    print(val[0],"\t",val[1],"\t",y[i])
    i+=1
print("="*20)

```

```

The input data is
VAR1    VAR2    CLASS
1.713    1.586    0
0.18     1.786    1
0.353    1.24     1
0.94     1.566    0
1.486    0.759    1
1.266    1.106    0
1.54     0.419    1
0.459    1.799    1
0.773    0.186    1
=====

```

```

print("The Test data to predict ")
test_data = []
VAR1 = float(input("Enter Value for VAR1 :"))
VAR2 = float(input("Enter Value for VAR2 :"))
test_data.append(VAR1)
test_data.append(VAR2)
print("="*20)
print("The predicted Class is : ",kmeans.predict([test_data]))

```

```

The Test data to predict
Enter Value for VAR1 :0.906
Enter Value for VAR2 :0.606
=====
The predicted Class is :  [0]

```

```

The input data is
VAR1    VAR2    CLASS
1.713    1.586    0
0.18     1.786    1
0.353    1.24     1
0.94     1.566    0
1.486    0.759    1
1.266    1.106    0
1.54     0.419    1
0.459    1.799    1
0.773    0.186    1
=====
The Test data to predict
Enter Value for VAR1 :0.906
Enter Value for VAR2 :0.606
=====
The predicted Class is : [0]

```

4.3 VIVA QUESTIONS & ANSWERS

1. What is K means Clustering Algorithm?

K Means algorithm is a centroid-based clustering (unsupervised) technique. This technique groups the dataset into k different clusters having an almost equal number of points. Each of the clusters has a centroid point which represents the mean of the data points lying in that cluster.

The idea of the K-Means algorithm is to find k-centroid points and every point in the dataset will belong to either of the k-sets having minimum Euclidean distance.

2. Is Feature Scaling required for the K means Algorithm?

Yes, K-Means typically needs to have some form of normalization done on the datasets to work properly since it is sensitive to both the mean and variance of the datasets.

For performing feature scaling, generally, **Standard Scaler** is recommended, but depending on the specific use cases, other techniques might be more suitable as well.

For Example, let's have 2 variables, named age and salary where age is in the range of 20 to 60 and salary is in the range of 100-150K, since scales of these variables are different so when these variables are substituted in the euclidean distance formula, then the variable which is on the large scale suppresses the variable which is on the smaller scale. So, the impact of age will not be captured very clearly. Hence, you have to scale the variables to the same range using **Standard Scaler**, **Min-Max Scaler**, etc.

3. Which metrics can you use to find the accuracy of the K means Algorithm?

There does not exist a correct answer to this question as K means being an unsupervised learning technique does not discuss anything about the output column. As a result, one can not get the accuracy number or values from the algorithm directly.

4. What are the advantages and disadvantages of the K means Algorithm?

Advantages:

- Easy to understand and implement.
- Computationally efficient for both training and prediction.
- Guaranteed convergence.

Disadvantages:

- We need to provide the number of clusters as an input variable to the algorithm.
- It is very sensitive to the initialization process.
- Good at clustering when we are dealing with spherical cluster shapes, but it will perform poorly when dealing with more complicated shapes.
- Due to the leveraging of the Euclidean distance function, it is sensitive to outliers.

5. What are the ways to avoid the problem of initialization sensitivity in the K means Algorithm?

There are two ways to avoid the problem of initialization sensitivity:

- **Repeat K means:** It basically repeats the algorithm again and again along with initializing the centroids followed by picking up the cluster which results in the small intracluster distance and large

intercluster distance.

- **KMeans++:** It is a smart centroid initialization technique.

Amongst the above two techniques, K-Means++ is the best approach.

PROGRAM5

5.TheFollowing Training ExamplesMap DescriptionsOfIndividualsOntoHigh,Medium And LowCredit-Worthiness.

mediumskiingdesignsingletwentiesno ->highRisk
high golftrading married forties yes ->lowRisk
lowspeedwaytransportmarriedthirtiesyes->medRisk
mediumfootballbanking single thirties yes ->lowRisk high
flying media married fifties yes ->highRisk
low football security single twenties no ->medRisk
medium golf media single thirties yes ->medRisk
medium golf transport married forties yes ->lowRisk
highskiingbankingsinglethirtiesyes->highRisklow
golf unemployed married forties yes ->highRisk

SOURCECODE:

Input attributes are (from left to right) income, recreation,job, status,age-group, home-owner.Findtheunconditionalprobabilityof golf'andtheconditionalprobabilityofsingle'given `medRisk' in thedataset?

totalRecords=10

numberGolfRecreation=4

probGolf=numberGolfRecreation/totalRecordsprint("Uncondition

al probability of golf: ={"}.format(probGolf)) #conditional

probability of `single' given `medRisk'

bayes Formula

$p(\text{single}|\text{medRisk})=p(\text{medRisk}|\text{single})p(\text{single})/p(\text{medRisk})$

$p(\text{medRisk}|\text{single})=p(\text{medRisk} \cap \text{single})/p(\text{single})$

#Therefore the result is:

numberMedRiskSingle=2

numberMedRisk=3

probMedRiskSingle=numberMedRiskSingle/totalRecordspr

obMedRisk=numberMedRisk/totalRecordsconditionalProba

bility=(probMedRiskSingle/probMedRisk)

print("Conditional probability of single given medRisk:={}".format(conditionalProbability))

OUTPUT:

Unconditional probability of golf:=0.4

Conditional probability of single given medRisk:=0.6666666666666667

Implementing in Code:

In the given data set,

The total number of records are 10.

The number of records which contains 'golf' are 4.

Then, the Unconditional probability of golf :

= The number of records which contains 'golf' / total number of records

= 4 / 10

= 0.4

To find the Conditional probability of single given medRisk,

Let S : single MR : medRisk

By the definition of Baye's rule(conditional probability), we have

$$P(S | MR) = P(S \cap MR) / P(MR)$$

$P(S \cap MR)$ = The number of MedRisk with Single records / total number of Records

$$= 2 / 10 = 0.2$$

$P(MR)$ = The number of records with MedRisk /total number of Records

$$= 3 / 10 = 0.3$$

The Conditional probability of single given medRisk

$$P(S | MR) = 0.2 / 0.3 = 0.66666$$

Code:

```
total_Records=10
numGolfRecords=4
unConditionalprobGolf=numGolfRecords / total_Records
print("Unconditional probability of golf: {}".format(unConditionalprobGolf))
#conditional probability of 'single' given 'medRisk'
numMedRiskSingle=2
numMedRisk=3
probMedRiskSingle=numMedRiskSingle/total_Records
probMedRisk=numMedRisk/total_Records
conditionalProb=(probMedRiskSingle/probMedRisk)
print("Conditional probability of single given medRisk: = {}".format(conditionalProb))
```

Output:

```
total_Records=10
numGolfRecords=4
unConditionalprobGolf=numGolfRecords / total_Records
print("Unconditional probability of golf: {}".format(unConditionalprobGolf))
```

Unconditional probability of golf: =0.4

```
numMedRiskSingle=2
numMedRisk=3
probMedRiskSingle=numMedRiskSingle/total_Records
probMedRisk=numMedRisk/total_Records
conditionalProb=(probMedRiskSingle/probMedRisk)
print("Conditional probability of single given medRisk: = {}".format(conditionalProb))
```

Conditional probability of single given medRisk: = 0.6666666666666667

```
Unconditional probability of golf: =0.4
Conditional probability of single given medRisk: = 0.6666666666666667
```

VIVA QUESTIONS & ANSWERS:

1. How Kmeans++ clustering Algorithm works?

KMeans++ algorithm is a smart technique for centroid initialization that initialized one centroid while ensuring the others to be far away from the chosen one resulting in faster convergence.

The steps to follow for centroid initialization are:

Step-1: Pick the first centroid point randomly.

Step-2: Compute the distance of all points in the dataset from the selected centroid. The distance of x_i point from the farthest centroid can be calculated by the given formula:

$$d_i = \max_{(j: 1 \rightarrow m)} \|x_i - C_j\|^2$$

where,

d_i : Distance of x_i point from the farthest centroid

m : number of centroids already picked

Step-3: Make the point x_i as the new centroid that is having maximum probability proportional to d_i

Step-4: Repeat the above last two steps till you find k centroids.

2. What is the training and testing complexity of the K means Algorithm?

Training complexity in terms of Big-O notation:

If we use Lloyd's algorithm, the complexity for training is: " $K * I * N * M$ "

where,

K: It represents the number of clusters

I: It represents the number of iterations **N:**

It represents the sample size

M: It represents the number of variables

Conclusion: There is a significant impact on capping the number of iterations.

Predicting complexity in terms of Big-O notation:

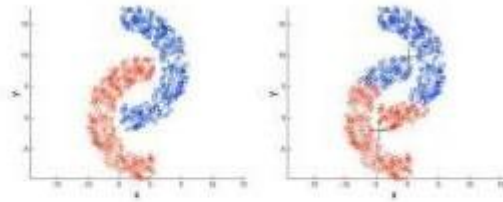
" $K * N * M$ "

Prediction needs to be computed for each record, the distance to each cluster and assigned to the nearest ones.

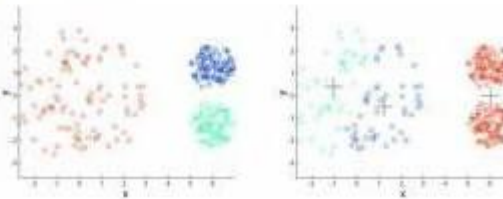
3. Is it possible that the assignment of data points to clusters does not change between successive iterations in the K means Algorithm?

When the K-Means algorithm has reached the local or global minima, it will not change the assignment of data points to clusters for two successive iterations during the algorithm run.

Non-convex/non-round-shaped clusters: Standard K -means fails!



Clusters with different densities



4. How to Choose K Value in K -Means:

1. Elbow method

steps:

step1: compute clustering algorithm for different values of k . for

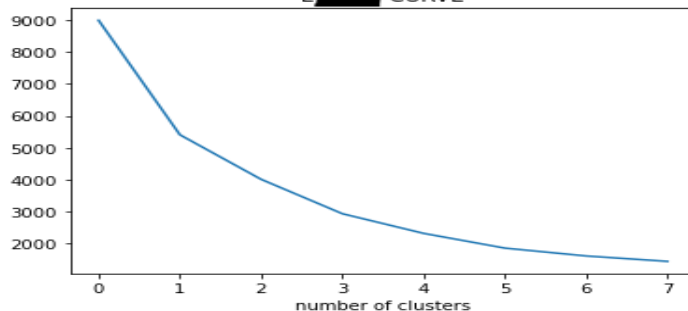
example $k=[1,2,3,4,5,6,7,8,9,10]$

step2: for each k calculate the within-cluster sum of squares (WCSS).

step3: plot curve of WCSS according to the number of clusters.

step4: The location of bend in the plot is generally considered an indicator of the approximate number of clusters.

$$WCSS = \sum (X_i - Y_i)^2$$



5. What are the practical considerations in K-Means:

- Choosing number of clusters in advance (K).
- Standardization of data (scaling).
- Categorical data (can be solved with K-Modes).
- Impact of initial centroids and outliers.

6. Why do you prefer Euclidean distance over Manhattan distance in the K-means algorithm?

Euclidean distance is preferred over Manhattan distance since Manhattan distance calculates distance only vertically or horizontally due to which it has dimension restrictions.

On the contrary, Euclidean distance can be used in any space to calculate the distances between the data points. Since in K-means algorithm the data points can be present in any dimension, so Euclidean distance is a more suitable option.

PROGRAM 6:

6.IMPLEMENTLINEARREGRESSIONUSINGPYTHON. AIM:

ToImplementlinearregressionusingpython.

DESCRIPTION:

Regression: Regression analysis is one of the most important fields in statistics and machine learning. There are many regression methods available. Linear regression is one of them.

WhatIsRegression?

Regression analysis is one of the most important fields in statistics and machine learning. There are many regression methods available. Linear regression is one of them. Regression searches for relationships among variables. For example, you can observe several employees of some company and try to understand how their salaries depend on the **features**, such as experience, level of education, role, city they work in, and so on. This is a regression problem where data related to each employee represent one **observation**. The presumption is that the experience, education, role, and city are the independent features, while the salary depends on them. Generally, in regression analysis, you usually consider some phenomenon of interest and there have a number of observations. Each observation has two or more features. Following the assumption that (at least) one of the features depends on the others, you try to establish a relation among them. You need to find a function that maps some features or variables to others sufficiently well. The dependent features are called the **dependent variables, outputs, or responses**. The independent features are called the **independent variables, inputs, or predictors**.

LinearRegression:

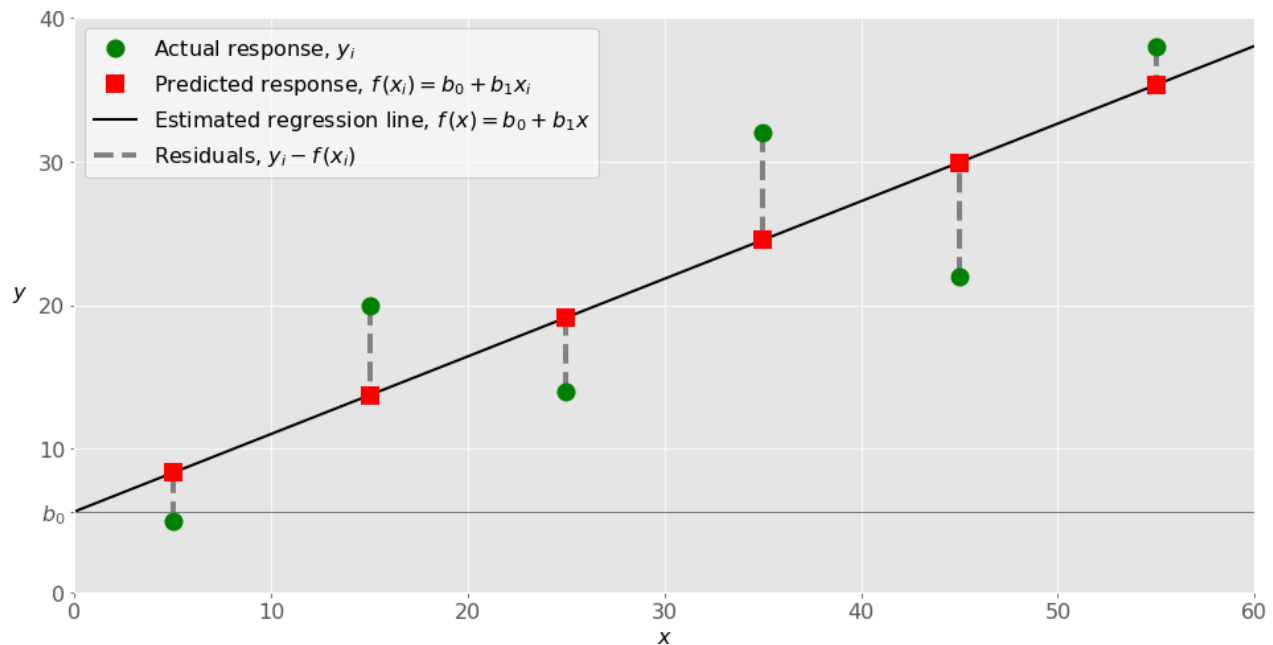
Linear regression is probably one of the most important and widely used regression techniques. It's among the simplest regression methods. One of its main advantages is the ease of interpreting results. When implementing linear regression of some dependent variable y on the set of independent variables $x = (x_1 \dots x_r)$, where r is the number of predictors, you assume a linear relationship between y and x : $y = \beta_0 + \beta_1 x_1 + \dots + \beta_r x_r + \epsilon$. This equation is the **regression equation**. β_0, β_r are the **regression coefficients**, and ϵ is the **random error**.

Linear regression calculates the **estimators** of the regression coefficients or simply the **predicted weights**, denoted with $b_1 \dots b_r$. They define the **estimated regression function** $f(x) = b_0 + b_1 x_1 +$

...+ $b_r x_r$. This function should capture the dependencies between the inputs and output sufficiently well.

Simple Linear Regression

The following figure illustrates simple linear regression:



When implementing simple linear regression, you typically start with a given set of input-output (x - y) pairs (green circles). These pairs are your observations. For example, the leftmost observation (green circle) has the input $x=5$ and the actual output (response) $y=5$. The next one has $x=15$ and $y=20$, and so on.

The estimated regression function (black line) has the equation $f(x) = b_0 + b_1 x$. Your goal is to calculate the optimal values of the predicted weights b_0 and b_1 that minimize SSR and determine the estimated regression function. The value of b_0 , also called the **intercept**, shows the point where the estimated regression line crosses the y -axis. It is the value of the estimated response $f(x)$ for $x=0$. The value of b_1 determines the **slope** of the estimated regression line.

The predicted responses (red squares) are the points on the regression line that correspond to the input values. For example, for the input $x=5$, the predicted response is $f(5) = 8.33$ (represented with the leftmost red square).

The residuals (vertical dashed gray lines) can be calculated as $y_i - f(x_i) = y_i - b_0 - b_1 x_i$ for $i=1, \dots, n$.

...,n . They are the distances between the green circles and red squares. When you implement linear regression, you are actually trying to minimize these distances and make the red squares as close to the predefined green circles as possible.

Implementing Linear Regression in Python

It's time to start implementing linear regression in Python. Basically, all you should do is apply the proper packages and their functions and classes.

Python Packages for Linear Regression

The package **NumPy** is a fundamental Python scientific package that allows many high-performance operations on single- and multi-dimensional arrays. It also offers many mathematical routines. Of course, it's open source.

The package **scikit-learn** is a widely used Python library for machine learning, built on top of NumPy and some other packages. It provides the means for preprocessing data, reducing dimensionality, implementing regression, classification, clustering, and more. Like NumPy, scikit-learn is also open source.

If you want to implement linear regression and need the functionality beyond the scope of scikit-learn, you should consider **statsmodels**. It's a powerful Python package for the estimation of statistical models, performing tests, and more. It's open source as well.

Simple Linear Regression With scikit-learn

Let's start with the simplest case, which is simple linear regression. There are

five basic steps when you're implementing linear regression:

1. Import the packages and classes you need.
2. Provide data to work with and eventually do appropriate transformations.
3. Create a regression model and fit it with existing data.
4. Check the results of model fitting to know whether the model is satisfactory.
5. Apply the model for predictions.

SOURCE CODE:

```
import numpy as np
import matplotlib.pyplot as plt

# ...

def estimate_coef(x, y):
    # number of observations/points n =
```

```

np.size(x)

#mean of x and y vector
m_x, m_y=np.mean(x),np.mean(y)

#calculating cross-deviation and deviation about x
SS_xy= np.sum(y*x)
- n*m_y*m_x

SS_xx=np.sum(x*x)-n*m_x*m_x

#calculating regression coefficients
b_1=
SS_xy/SS_xx

b_0=m_y-

b_1*m_x
return(b_0,

b_1)

def plot_regression_line(x,y,b):

#plotting the actual points as scatter plot
plt.scatter(x,
y, color = "m",
marker= "o",s= 30)

#predicted response vector
y_pred=
b[0] + b[1]*x

# plotting the regression line
plt.plot(x,y_pred,color="g")

#putting labels
plt.xlabel('x')

plt.ylabel('y')

```

```

#function to show plot
plt.show()

def main():
    #observations
    x=np.array([0,1, 2, 3, 4, 5,6, 7, 8,9])
    y=np.array([1,3, 2, 5, 7, 8,8, 9, 10,12])

    #estimating coefficients
    b=estimate_coef(x, y)
    print("Estimated coefficients:\nb_0={}\bnb_1={}".format(b[0],b[1]))

    # plotting regression line
    plot_regression_line(x,y,b)

if __name__ == "__main__":
    main()

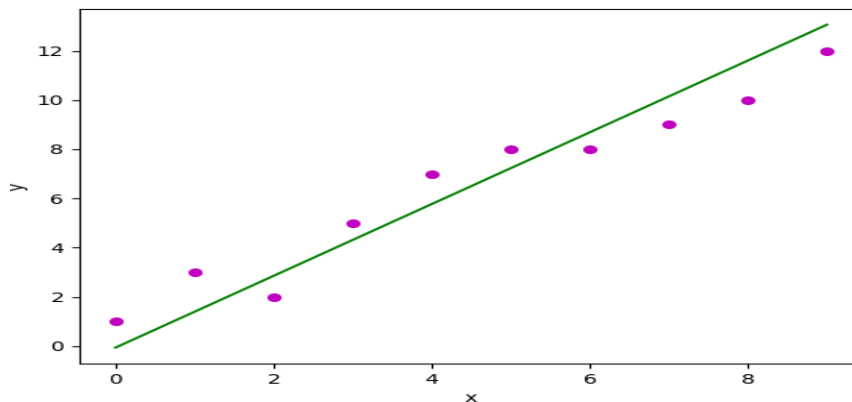
```

OUTPUT:

```

Estimated coefficients:
b_0=-0.05862068965

```



VIVA QUESTIONS AND ANSWERS

1. What do you mean by the Logistic Regression?

It's a classification algorithm that is used where the target variable is of categorical nature. The main objective behind Logistic Regression is to determine the relationship between features and the probability of a particular outcome.

For Example, when we need to predict whether a student passes or fails in an exam given the number of hours spent studying as a feature, the target variable comprises two values i.e. pass and fail.

Therefore, we can solve classification problem statements which is a supervised machine learning technique using Logistic Regression.

2. What are the different types of Logistic Regression?

Three different types of Logistic Regression are as follows:

1. Binary Logistic Regression: In this, the target variable has only two possible outcomes.

For Example, 0 and 1, or pass and fail or true and false.

2. Multinomial Logistic Regression: In this, the target variable can have three or more possible values without any order.

For Example, Predicting preference of food i.e. Veg, Non-Veg, Vegan.

3. Ordinal Logistic Regression: In this, the target variable can have three or more values with ordering.

For Example, Movie rating from 1 to 5.

3. How do we handle categorical variables in Logistic Regression?

The inputs given to a Logistic Regression model need to be numeric. The algorithm cannot handle categorical variables directly. So, we need to convert the categorical data into a numerical format that is suitable for the algorithm to process.

Each level of the categorical variable will be assigned a unique numeric value also known as a **dummy variable**. These dummy variables are handled by the Logistic Regression model in the same manner as any other numeric value.

4. What are the assumptions made in Logistic Regression?

Some of the assumptions of Logistic Regression are as follows:

1. It assumes that there is **minimal or no multicollinearity** among the independent variables i.e., predictors are not correlated.
2. There should be a linear relationship between the logit of the outcome and each predictor variable. The logit function is described as **$\text{logit}(p) = \log(p/(1-p))$** , where p is the probability of the target outcome.
3. Sometimes to predict properly, it usually requires a **large sample size**.
4. The Logistic Regression which has **binary classification** i.e., two classes assumes that the target variable is binary, and ordered Logistic Regression requires the target variable to be ordered.

For example, Too Little, About Right, Too Much.

5. It assumes there is **no dependency** between the observations.

5. Can we solve the multiclass classification problems using Logistic Regression? If Yes then How?

Yes, in order to deal with multiclass classification using Logistic Regression, the most famous method is known as the one-vs-all approach. In this approach, a number of models are trained, which is equal to the number of classes. These models work in a specific way.

For Example, the first model classifies the data point depending on whether it belongs to class 1 or some other class (not class 1); the second model classifies the data point into class 2 or some other class (not class 2) and so on for all other classes. So, in this manner, each data point can be checked over all the classes.

PROGRAM7

IMPLEMENT NAÏVE BAYES THEOREM TO CLASSIFY THE ENGLISH TEXT

AIM:

To implement naïve Bayes's theorem to classify the English text

DESCRIPTION:

The challenge of text classification is to attach labels to bodies of text, e.g., tax document, medical form, etc. based on the text itself. For example, think of your spam folder in your email. How does your email provider know that a particular message is spam or “ham” (not spam)? We'll take a look at one natural language processing technique for text classification called Naive Bayes

SOURCE CODE:

```
import pandas as pd

from sklearn.model_selection import

train_test_split from sklearn.feature_extraction.text import

CountVectorizer from sklearn.naive_bayes import

MultinomialNB

from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score

msg = pd.read_csv('document.csv', names=['message', 'label'])

print("Total Instances of Dataset: ", msg.shape[0])

msg['labelnum'] = msg.label.map({'pos': 1, 'neg': 0})

X = msg.message

y = msg.labelnum

X_train, X_test, y_train, y_test = train_test_split(X, y)

count_v = CountVectorizer()
```

```

Xtrain_dm= count_v.fit_transform(Xtrain)

Xtest_dm = count_v.transform(Xtest)

df=pd.DataFrame(Xtrain_dm.toarray(),columns=count_v.get_feature_names())

clf = MultinomialNB()

clf.fit(Xtrain_dm, ytrain)

pred=clf.predict(Xtest_dm)

print('AccuracyMetrics:')

print('Accuracy:',accuracy_score(ytest,pred))print('Recall:',recall_score(ytest,pred))print('Precision:',
precision_score(ytest, pred))

print('ConfusionMatrix:\n',confusion_matrix(ytest,pred))

```

document.csv:

```

Ilovethissandwich,posThis is
an amazingplace,pos
Ifeelverygoodaboutthesebeers,pos
This is my best work,pos
Whatanawesomeview,pos
Idonotlikethisrestaurant,neg I
am tired of this stuff,neg
Ican'tdealwiththis,negHe
ismyswornenemy,negMy

```

bossishorrible,neg

Thisisanawesomeplace,pos

Ido notlikethetasteofthisjuice,neg I

love to dance,pos

Iamsickandtiredofthisplace,neg What

a great holiday,pos

Thatisabadlocalitytostay,neg

Wewillhavegoodfuntomorrow,posI went

to my enemy's house today,neg

OUTPUT:

TotalInstancesofDataset:18

AccuracyMetrics:

Accuracy:0.6

Recall:0.6666666666666666

Precision:0.6666666666666666

ConfusionMatrix:

[[1 1]

[12]]

VIVA QUESTIONS & ANSWERS

1. How Naive Bayes algorithm works?

Let's understand it using an example. Below I have a training dataset of weather and corresponding target variable 'Play' (suggesting possibilities of playing). Now, we need to classify whether players will play or not based on weather condition. Let's follow the below steps to perform it.

Step 1: Convert the dataset into a frequency table

Step 2: Create Likelihood table by finding the probabilities like Overcast probability = 0.29 and probability of playing is 0.64.

Weather	Play
Sunny	No
Overcast	Yes
Rainy	Yes
Sunny	Yes
Sunny	Yes
Overcast	Yes
Rainy	No
Rainy	No
Sunny	Yes
Rainy	Yes
Sunny	No
Overcast	Yes
Overcast	Yes
Rainy	No

Frequency Table		
Weather	No	Yes
Overcast		4
Rainy	3	2
Sunny	2	3
Grand Total	5	9

Likelihood table				
Weather	No	Yes		
Overcast		4	=4/14	0.29
Rainy	3	2	=5/14	0.36
Sunny	2	3	=5/14	0.36
All	5	9		
	=5/14	=9/14		
	0.36	0.64		

Step 3: Now, use Naive Bayes equation to calculate the posterior probability for each class. The class with the highest posterior probability is the outcome of prediction.

Problem: Players will play if weather is sunny. Is this statement correct? We

can solve it using above discussed method of posterior probability.

$$P(\text{Yes} | \text{Sunny}) = P(\text{Sunny} | \text{Yes}) * P(\text{Yes}) / P(\text{Sunny})$$

Here we have $P(\text{Sunny} | \text{Yes}) = 3/9 = 0.33$, $P(\text{Sunny}) = 5/14 = 0.36$, $P(\text{Yes}) = 9/14 = 0.64$ Now, P

$$(\text{Yes} | \text{Sunny}) = 0.33 * 0.64 / 0.36 = 0.60, \text{ which has higher probability.}$$

Naive Bayes uses a similar method to predict the probability of different class based on various attributes. This algorithm is mostly used in text classification and with problems having multiple classes.

2. Applications of Naive Bayes Algorithms

- **Real time Prediction:** Naive Bayes is an eager learning classifier and it is sure fast. Thus, it could be used for making predictions in real time.
- **Multi class Prediction:** This algorithm is also well known for multi class prediction feature. Here we can predict the probability of multiple classes of target variable.
- **Text classification/ Spam Filtering/ Sentiment Analysis:** Naive Bayes classifiers mostly used in text classification (due to better result in multiclass problems and independence rule) have higher success rates as compared to other algorithms. As a result, it is widely used in Spam filtering (identify spam e-mail) and Sentiment Analysis (in social media analysis, to identify positive and negative customer sentiments)
- **Recommendation System:** Naive Bayes Classifier and Collaborative Filtering together builds a Recommendation System that uses machine learning and data mining techniques to filter unseen information and predict whether a user would like a given resource or not

PROGRAM 8

8. IMPLEMENT AN ALGORITHM TO DEMONSTRATE THE SIGNIFICANCE OF GENETICAL ALGORITHM

AIM:

To implement an algorithm to demonstrate the significance of genetic algorithm

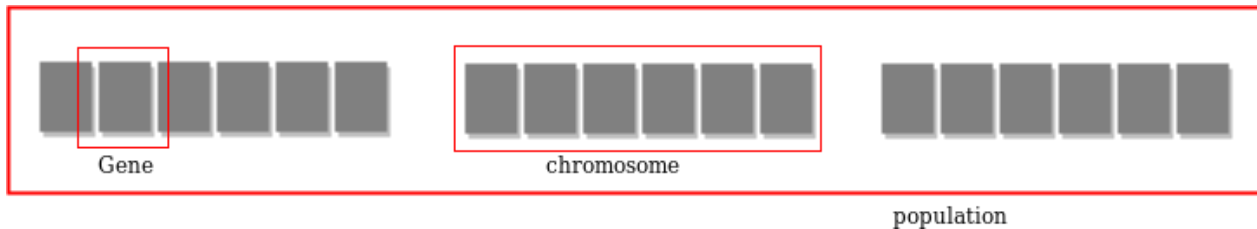
DESCRIPTION:

Genetic Algorithms (GAs) are adaptive heuristic search algorithms that belong to the larger part of evolutionary algorithms. Genetic algorithms are based on the ideas of natural selection and genetics. These are intelligent exploitation of random search provided with historical data to direct the search into the region of better performance in solution space. They are commonly used to generate high-quality solutions for optimization problems and search problems. Genetic algorithms simulate the process of natural selection which means those species who can adapt to changes in their environment are able to survive and reproduce and go to next generation. In simple words, they simulate “survival of the fittest” among individual of consecutive generation for solving a problem. Each generation consists of a population of individuals and each individual represents a point in search space and possible solution. Each individual is represented as a string of character/integer/float/bits. This string is analogous to the Chromosome. Genetic algorithms are based on an analogy with genetic structure and behavior of chromosome of the population. Following is the foundation of GAs based on this analogy –

- Individual in population competes for resources and mate
- Those individuals who are successful (fittest) then mate to create more offspring than others
- Genes from “fittest” parent propagate throughout the generation, that is sometimes parents create offspring which is better than either parent.
- Thus each successive generation is more suited for their environment.

The population of individuals are maintained within search space. Each individual represents a solution in search space for given problem. Each individual is coded as a finite length vector (analogous to chromosome) of components. These variable components are

analogous to Genes. Thus a chromosome (individual) is composed of several genes (variable components)

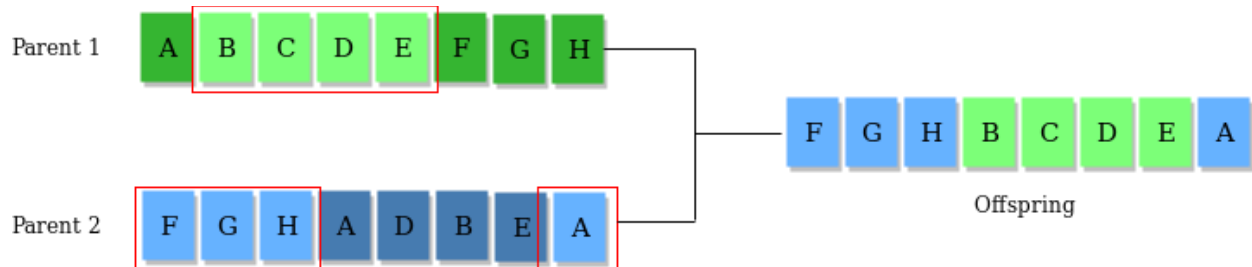


A Fitness Score is given to each individual which shows the ability of an individual to “compete”. The individual having optimal fitness score (or near optimal) are sought. The GAs maintain the population of n individuals (chromosome/solutions) along with their fitness scores. The individuals having better fitness scores are given more chance to reproduce than others. The individuals with better fitness scores are selected who mate and produce **better offspring** by combining chromosomes of parents. The population size is static so the room has to be created for new arrivals. So, some individuals die and get replaced by new arrivals eventually creating new generation when all the mating opportunity of the old population is exhausted. It is hoped that over successive generations better solutions will arrive while least fit die. Each new generation has on average more “better genes” than the individual (solution) of previous generations. Thus each new generation has better “**partial solutions**” than previous generations. Once the offsprings produced having no significant difference than offspring produced by previous populations, the population is converged. The algorithm is said to be converged to a set of solutions for the problem.

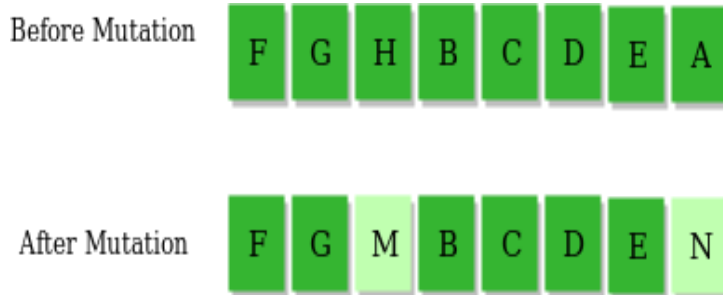
OPERATORS OF GENETICAL ALGORITHM:

Once the initial generation is created, the algorithm evolves the generation using following operators –

- 1) **Selection Operator:** The idea is to give preference to the individuals with good fitness scores and allow them to pass their genes to the successive generations.
- 2) **Crossover Operator:** This represents mating between individuals. Two individuals are selected using selection operator and crossover sites are chosen randomly. Then the genes at these crossover sites are exchanged thus creating a completely new individual (offspring). For example –



Mutation Operator: The key idea is to insert random genes in offspring to maintain the diversity in population to avoid the premature convergence. For example –



ALGORITHM:

- 1) Randomly initialize population sp
- 2) Determine fitness of population
- 3) Until convergence repeat:
 - a) Select parents from population
 - b) Crossover and generate new population
 - c) Perform mutation on new population
 - d) Calculate fitness for new population

USES OF GENETIC ALGORITHM:

- They are Robust
- Provide optimisation over large space state.
- Unlike traditional AI, they do not break on slight change in input or presence of noise

SOURCECODE:

```
import numpy as np
def cal_pop_fitness(equation_inputs, pop):
    # Calculating the fitness value of each solution in the current population.
    # The fitness function calculates the sum of products between each input and its corresponding weight.
    fitness = numpy.sum(pop * equation_inputs, axis=1)
    return fitness

def select_mating_pool(pop, fitness, num_parents):
    # Selecting the best individuals in the current generation as parents for producing the offspring of the next generation.
    parents = numpy.empty((num_parents, pop.shape[1]))
    for parent_num in range(num_parents):
        max_fitness_idx = numpy.where(fitness == numpy.max(fitness))
        max_fitness_idx = max_fitness_idx[0][0]
        parents[parent_num, :] = pop[max_fitness_idx, :]
        fitness[max_fitness_idx] = -999999999999
    return parents

def crossover(parents, offspring_size):
    offspring = numpy.empty(offspring_size)
```

```

#The point at which crossover takes place between two parents. Usually, it is at the center. crossover_point =
numpy.uint8(offspring_size[1]/2)

for k in range(offspring_size[0]):
    #Index of the first parent to
    mate. parent1_idx =
    k%parents.shape[0]
    # Index of the second parent to mate.
    parent2_idx=(k+1)%parents.shape[0]
    #Thenew offspring will have its first half of its genes taken from the first parent.
    offspring[k, 0:crossover_point] = parents[parent1_idx, 0:crossover_point]
    #Thenew offspring will have its second half of its genes taken from the second parent. offspring[k,
    crossover_point:] = parents[parent2_idx, crossover_point:]
    return offspring

def mutation(offspring_crossover, num_mutations=1):
    mutations_counter=numpy.uint8(offspring_crossover.shape[1]/num_mutations)
    #Mutation changes a number of genes as defined by the num_mutations
    argument. The changes are random.
    for idx in
    range(offspring_crossover.shape[0]):
        gene_idx = mutations_counter - 1
        for mutation_num in range(num_mutations): #
            The random value to be added to the gene.

```

```

random_value=numpy.random.uniform(-1.0,1.0, 1)

offspring_crossover[idx, gene_idx] = offspring_crossover[idx,
gene_idx] + random_value

gene_idx = gene_idx +
mutations_counterreturn
offspring_crossover

importnumpy

```

"""

The y=target is to maximize this

equation ASAP: $y =$

$w_1x_1+w_2x_2+w_3x_3+w_4x_4+w_5x_5+6wx_6$

where(x_1,x_2,x_3,x_4,x_5,x_6)=(4,-2,3.5,5,-

11,-4.7)

What are the best values for the 6 weights w_1 to w_6 ?

We are going to use the genetic algorithm for the best possible values after a number of generations.

"""

#Input of the equation.

equation_inputs = [4,-

2,3.5,5,-11,-4.7]

Number of the weights we are looking to

optimize.num_weights=len(equation_inputs)

```
"""
```

```
Genetic algorithm
```

```
parameters:Mating
```

```
pool size
```

```
Population size
```

```
"""
```

```
sol_per_pop=8
```

```
num_parents_mating= 4
```

```
#Definingthepopulationsize.
```

```
pop_size=(sol_per_pop,num_weights)#Thepopulationwillhavesol_per_pop  
chromosome where each chromosome has num_weights genes.
```

```
#Creatingthe initialpopulation.
```

```
new_population=numpy.random.uniform(low=-4.0,high=4.0,
```

```
size=pop_size) print(new_population)
```

```
"""
```

```
new_population[0,:]=[2.4,0.7,8,-2,5,1.1]
```

```
new_population[1,:]=[-0.4,2.7,5,-1,7,0.1]
```

```
new_population[2,:]=[-1,2,2,-3,2,0.9]
```

```
new_population[3,:]=[4,      7,12,6.1,1.4,-4]
```

```
new_population[4,:]=[3.1,4,0,2.4, 4.8,0]
```

```
new_population[5,:]=[-2,3,-
```

```
7,6,3,      3]"""
```

```

best_outputs = []

num_generations=

1000

for generation in

range(num_generations):

print("Generation:",

generation)

# Measuring the fitness of each chromosome in the

population.fitness=cal_pop_fitness(equation_inputs,

new_population) print("Fitness")

print(fitness)

best_outputs.append(numpy.max(numpy.sum(new_population*equation_inputs,

axis=1)))#Thebestresultinthecurrentiteration.

print("Bestresult:",numpy.max(numpy.sum(new_population*equation_inputs,axis=1)))

#Selectingthebest parentsinthepopulationfor

mating. parents =

select_mating_pool(new_population, fitness,

num_parents_mating)

print("Parents

")

print(parents)

#Generatingnextgenerationusing

```

```

crossover.offspring_crossover=
crossover(parents,
offspring_size=(pop_size[0]-parents.shape[0],num_weights))

print("Crossover")
print(offspring_cr
ossover)

#Addingsomevariationstotheoffspringusingmutation.
offspring_mutation = mutation(offspring_crossover,
num_mutations=2) print("Mutation")
print(offspring_mutation)

#Creatingthenewpopulationbasedontheparentsandoffspring.
new_population[0:parents.shape[0], :] = parents
new_population[parents.shape[0]:, :] = offspring_mutation

#Gettingthebestsolutionafteriteratingfinishingallgenerations. #At
first, the fitness is calculated for each solution in the final
generation. fitness = cal_pop_fitness(equation_inputs,
new_population)

#Thenreturnthe indexofthat solutioncorrespondingtothe best
fitness. best_match_idx = numpy.where(fitness ==
numpy.max(fitness))

print("Bestsolution:",

```

```

new_population[best_match_idx,:])print("Best
solution fitness : ", fitness[best_match_idx])
importmatplotlib.pyplotmatplotlib.pyplot.plot(b
est_outp
uts)
matplotlib.pyplot.xlabel("Ite
ration")
matplotlib.pyplot.ylabel("Fit
ness")
matplotlib.pyplot.show()

```

OUTPUT:

```

[[0.582041412.32880696-2.951302092.570569533.33055238-0.58167871] [-1.65052225
3.52263842-2.46577305-1.7005396-3.804802020.29677167] [ 2.6239874-2.01548549-
1.722922953.61090243-1.25604726-2.32647264][-3.451673932.857718253.74655682-
2.017906260.25750106-3.12923247][2.86026334-0.4306777-3.262979561.74863348-
1.93705571-3.18855672][-1.700120890.98685104-1.911920723.91873942-0.09354385
1.43038667] [0.31769009-0.872908093.752497852.576579930.588830822.83231871][
3.833149260.33838112-2.49509594-1.507631743.99440509-0.03037715]]

```

Gen

erati

on :

0

Fitness

s

```

[-33.708344139.6777259451.30214363-4.6238336545.91897711-1.566046069.24418172-

```

45.41084308]

Best result:

51.302143629097614

Parents

[[2.6239874-2.01548549-1.722922953.61090243-1.25604726-2.32647264][2.86026334-0.4306777-3.262979561.74863348-1.93705571-3.18855672][-1.650522253.52263842-2.46577305-1.7005396-3.804802020.29677167][0.31769009-0.872908093.75249785 2.576579930.58883082 2.83231871]]

Crossover

[[2.6239874-2.01548549-1.722922951.74863348-1.93705571-3.18855672][2.86026334-0.4306777-3.26297956-1.7005396-3.804802020.29677167] [-1.650522253.52263842-2.46577305 2.576579930.588830822.83231871][0.31769009-0.872908093.75249785 3.61090243-1.25604726-2.32647264]]

Mutation

[[2.6239874-2.01548549-1.678966321.74863348-1.93705571-3.97789372][2.86026334-0.4306777-3.12878279-1.7005396-3.80480202-0.15430324][-1.650522253.52263842-3.37669601 2.576579930.588830822.25153466][0.31769009-0.872908092.93428907 3.61090243-1.25604726-2.71597954]]

.

.

.

.

Gener

ation:

999

Fitness

s

[2554.39355622551.72360738 2549.405839542549.299316292552.24225166
2550.45506206
2547.12995122551.22467397]

Best result:

2554.3935561987346

Parents

[[3.17690088e-01-8.72908094e-012.67689952e+021.74863348e+00-
1.93705571e+00 -3.37108802e+02][3.17690088e-01 -8.72908094e-01
2.67638232e+021.74863348e+00-
1.93705571e+00-3.36689592e+02][3.17690088e-01-8.72908094e-012.67254110e+02
1.74863348e+00-1.93705571e+00-3.36865291e+02][3.17690088e-01-8.72908094e-01
2.67370854e+021.74863348e+00-1.93705571e+00-3.36672197e+02]]

Crossover

[[3.17690088e-01-8.72908094e-012.67689952e+021.74863348e+00-
1.93705571e+00 - 3.36689592e+02]

[3.17690088e-01-8.72908094e-012.67638232e+021.74863348e+00
-1.93705571e+00-3.36865291e+02]

[3.17690088e-01-8.72908094e-012.67254110e+021.74863348e+00
-1.93705571e+00-3.36672197e+02]

[3.17690088e-01-8.72908094e-012.67370854e+021.74863348e+00
-1.93705571e+00-3.37108802e+02]]

Mutation

[[3.17690088e-01-8.72908094e-01 2.68382875e+021.74863348e+00
-1.93705571e+00-3.36222272e+02]

[3.17690088e-01-8.72908094e-012.68456819e+021.74863348e+00

-1.93705571e+00-3.37417363e+02]

[3.17690088e-01-8.72908094e-012.67606746e+021.74863348e+00

-1.93705571e+00-3.36866918e+02]

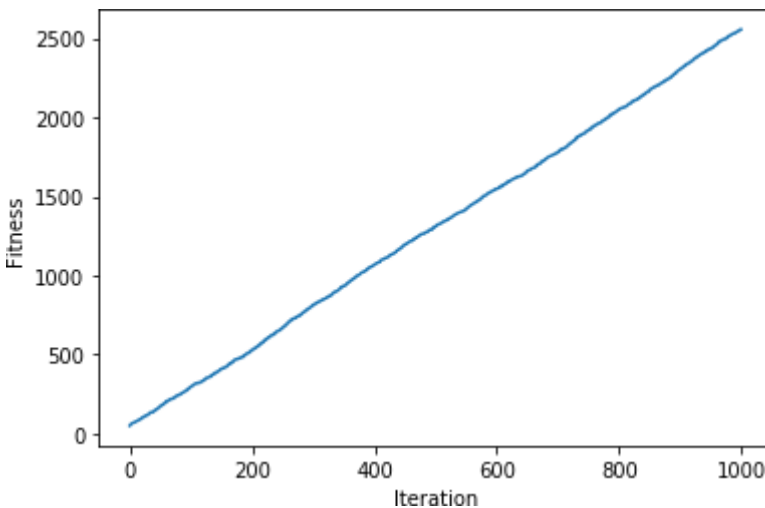
[3.17690088e-01-8.72908094e-012.67051753e+021.74863348e+00

-1.93705571e+00-3.37331663e+02]]

Bestsolution: [[[3.17690088e-01-8.72908094e-012.68456819e+021.74863348e+00

-1.93705571e+00-
3.37417363e+02]]]

Bestsolutionfitness
:[2558.52782726]



VIVA QUESTIONS & ANSWERS

1. What is a Genetic Algorithm?

Let's get back to the example we discussed above and summarize what we did.

1. Firstly, we defined our initial population as our countrymen.
2. We defined a function to classify whether a person is good or bad.

3. Then we selected good people for mating to produce their off-springs.
4. And finally, these off-springs replace the bad people from the population and this process repeats.

PROGRAM 9

9. IMPLEMENT THE FINITE WORDS CLASSIFICATION SYSTEM USING BACK-PROPAGATIONAL ALGORITHM

AIM:

To implement the finite words classification system using Back-propagation algorithm

DESCRIPTION:

What is backpropagation?

We can define the backpropagation algorithm as an algorithm that trains some given feed-forward Neural Network for a given input pattern where the classifications are known to us. At the point when every passage of the example set is exhibited to the network, the network looks at its yield reaction to the example input pattern. After that, the comparison done between output response and expected output with the error value is measured. Later, we adjust the connection weight based upon the error value measured.

It was first introduced in the 1960s and 30 years later it was popularized by David Rumelhart, Geoffrey Hinton, and Ronald Williams in the famous 1986 paper. In this paper, they spoke about the various neural networks. Today, back propagation is doing good. Neural network training happens through back propagation. By this approach, we fine-tune the weights of a neural network based on the error rate obtained in the previous run. The right manner of applying this technique reduces error rates and makes the model more reliable. Backpropagation is used to train the neural network of the chain rule method. In simple terms, after each feed-forward pass through a network, this algorithm does the backward

pass to adjust the model's parameters based on weights and biases. A typical supervised learning algorithm attempts to find a function that maps input data to the right output. Back propagation works with a multi-layered neural network and learns internal representations of input to output mapping.

The Back propagation algorithm is a supervised learning method for multilayer feed-forward networks from the field of Artificial Neural Networks.

Feed-forward neural networks are inspired by the information processing of one or more neural cells, called a neuron. A neuron accepts input signals via its dendrites, which pass the electrical signal down to the cell body. The axon carries the signal out to synapses, which are the connections of a cell's axon to other cell's dendrites.

The principle of the backpropagation approach is to model a given function by modifying internal weightings of input signals to produce an expected output signal. The system is trained using a supervised learning method, where the error between the system's output and a known expected output is presented to the system and used to modify its internal state.

Technically, the backpropagation algorithm is a method for training the weights in a multilayer feed-forward neural network. As such, it requires a network structure to be defined of one or more layers where one layer is fully connected to the next layer. A standard network structure is one input layer, one hidden layer, and one output layer. Back propagation can be used for both classification and regression problems.

In classification problems, best results are achieved when the network has one neuron in the output layer for each class value. For example, a 2-class or binary classification problem with the class values of A and B. These expected outputs would have to be transformed into binary vectors with one column for each class value. Such as $[1, 0]$ and $[0, 1]$ for A and B respectively. This is called a one hot encoding.

How does backpropagation work?

Let us take a look at how backpropagation works. It has four layers: input layer, hidden layer, hidden layer II and final output layer.

So, the main three layers are:

1. Input layer
2. Hidden layer
3. Output layer

Each layer has its own way of working and its own way to take action such that we are able to get the desired results and correlate these scenarios to our conditions. Let us discuss other details needed to help summarize this algorithm.

SOURCE CODE:

```
import pandas as pd

from sklearn.model_selection import

train_test_split from sklearn.feature_extraction.text

import CountVectorizer from

sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score

msg = pd.read_csv('document.csv',

names=['message', 'label']) print("Total Instances of

Dataset: ", msg.shape[0]) msg['labelnum'] =

msg.label.map({'pos': 1, 'neg': 0})

X=

msg.me

ssage y
```

```

=
msg.lab
enum

Xtrain,Xtest,ytrain,ytest=train_test_split(X,y)

count_v = CountVectorizer()

Xtrain_dm =
count_v.fit_transform(Xtrain)

Xtest_dm=
count_v.transform(Xtest)

df=pd.DataFrame(Xtrain_dm.toarray(),columns=count_v.get_feature_names())

clf=MLPClassifier(solver='lbfgs',alpha=1e-5,hidden_layer_sizes=(5,2),random_state=1)

clf.fit(Xtrain_dm, ytrain)pred=clf.predict(Xtest_dm)

print('AccuracyMetrics:')

    print('Accuracy:',accuracy_score(ytest,pred))print('Recall:',recall_score(ytest,pred))
print('Precision: ',      precision_score(ytest, pred))

print('ConfusionMatrix:\n',confusion_matrix(ytest,pred))

document.csv:

I love this

sandwich,posThis

is an

amazingplace,pos

```

I feel very good about these

beers, pos This is my best

work, pos

What an awesome view, pos I

do not like this

restaurant, neg I am

tired of this stuff, neg I

can't deal with

this, neg He is

my sworn

enemy, neg My

boss is

horrible, neg

This is an awesome place, pos I

do not like the taste of

this juice, neg I love to

dance, pos

I am sick and tired of this place, neg

What a great holiday, pos

That is a bad locality to stay, neg We
will have good fun

tomorrow, pos I went to my

enemy'shousetoday,neg

OUTPUT:

TotalInstances of

Dataset: 18

AccuracyMetrics:

Accuracy:0.8

Recall:1.0

Precisio

n:0.75

Confusi

on

Matrix:

[[11]

[03]

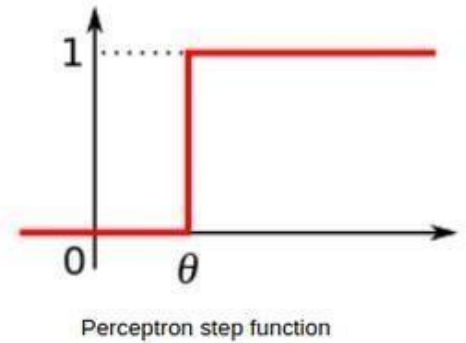
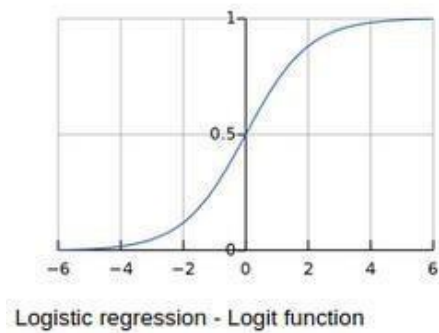
VIVA QUESTIONS & ANSWERS:

1. What is the difference between a Perceptron and Logistic Regression?

A Multi-Layer Perception (MLP) is one of the most basic neural networks that we use for classification. For a binary classification problem, we know that the output can be either 0 or 1. This is just like our simple logistic regression, where we use a logit function to generate a probability between 0 and 1.

So, what's the difference between the two?

Simply put, it is just the difference in the threshold function! When we restrict the logistic regression model to give us either exactly 1 or exactly 0, we get a Perceptron model:



2. Can we have the same bias for all neurons of a hidden layer?

Essentially, you can have a different bias value at each layer or at each neuron as well. However, it is best if we have a bias matrix for all the neurons in the hidden layers as well.

A point to note is that both these strategies would give you very different results.

3. What if we do not use any activation function(s) in a neural network?

The main aim of this question is to understand why we need activation functions in a neural network. You can start off by giving a simple explanation of how neural networks are built:

Step 1: Calculate the sum of all the inputs (X) according to their weights and include the bias term:

$$Z = (\text{weights} * X) + \text{bias}$$

Step 2: Apply an activation function to calculate the expected output: $Y =$

$$\text{Activation}(Z)$$

Steps 1 and 2 are performed at each layer. If you recollect, this is nothing but forward propagation! Now, what if there is no activation function?

Our equation for Y essentially becomes:

$$Y = Z = (\text{weights} * X) + \text{bias}$$

Wait – isn't this just a simple linear equation? Yes – and that is why we need activation functions. A linear equation will not be able to capture the complex patterns in the data – this is even more evident in the case of deep learning problems.

In order to capture non-linear relationships, we use activation functions, and that is why a neural network without an activation function is just a linear regression model.

4. In a neural network, what if all the weights are initialized with the same value?

In simplest terms, if all the neurons have the same value of weights, each hidden unit will get exactly the same signal. While this might work during forward propagation, the derivative of the cost function during backward propagation would be the same everytime.

In short, there is no learning happening by the network! What do you call the phenomenon of the model being unable to learn any patterns from the data? Yes, under fitting.

Therefore, if all weights have the same initial value, this would lead to underfitting.

5. What is the role of weights and bias in a neural network?

This is a question best explained with a real-life example. Consider that you want to go out today to play a cricket match with your friends. Now, a number of factors can affect your decision-making, like:

- How many of your friends can make it to the game?
- How much equipment can all of you bring?
- What is the temperature outside?

And so on. These factors can change your decision greatly or not too much. For example, if it is raining outside, then you cannot go out to play at all. Or if you have only one bat, you can share it while playing as well. The magnitude by which these factors can affect the game is called the weight of that factor.

Factors like the weather or temperature might have a higher weight, and other factors like equipment would have a lower weight.

However, does this mean that we can play a cricket match with only one bat? No – we would need 1 ball and 6 wickets as well. This is where bias comes into the picture. Bias lets you assign some threshold which helps you activate a decision-point (or a neuron) only when that threshold is crossed.

PROGRAM 10 (a)

Aim: Build SVM model with various kernels and select the best kernel for given dataset.

Software Required: Google CoLab, Jupyter notebook, Kaggle.

Dataset: <https://www.kaggle.com/datasets/uciml/human-activity-recognition-with-smartphones>

About Dataset The Human Activity Recognition database was built from the recordings of 30 study participants performing activities of daily living (ADL) while carrying a waist-mounted smartphone with embedded inertial sensors. The objective is to classify activities into one of the six activities performed.

Description of experiment The experiments have been carried out with a group of 30 volunteers within an age bracket of 19-48 years. Each person performed six activities (WALKING, WALKINGUPSTAIRS, WALKINGDOWNSTAIRS, SITTING, STANDING, LAYING) wearing a

smartphone (Samsung Galaxy S II) on the waist. Using its embedded accelerometer and gyroscope, we captured 3-axial linear acceleration and 3-axial angular velocity at a constant rate of 50Hz. The experiments have been video-recorded to label the data manually. The obtained dataset has been randomly partitioned into two sets, where 70% of the volunteers was selected for generating the training data and 30% the test data.

The sensor signals (accelerometer and gyroscope) were pre-processed by applying noise filters and then sampled in fixed-width sliding windows of 2.56 sec and 50% overlap (128 readings/window). The sensor acceleration signal, which has gravitational and body motion components, was separated using a Butterworth low-pass filter into body acceleration and gravity. The gravitational force is assumed to have only low frequency components, therefore a filter with 0.3 Hz cutoff frequency was used. From each window, a vector of features was obtained by calculating variables from the time and frequency domain.

Attribute information

For each record in the dataset the following is provided:

Triaxial acceleration from the accelerometer

(total acceleration) and the estimated body acceleration. Triaxial Angular velocity from the gyroscope.

A 561-feature vector

with time and frequency domain variables. Its activity label.

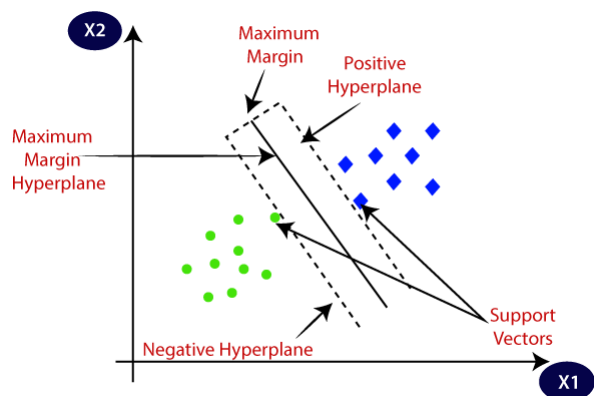
An identifier of the subject who carried out the experiment.

Theory:

Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems. However, primarily, it is used for Classification problems in Machine Learning.

The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n -dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.

SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:



Example: SVM can be understood with the example that we have used in the KNN classifier. Suppose we see a strange cat that also has some features of dogs, so if we want a model that can accurately identify whether it is a cat or dog, so such a model can be created by using the SVM algorithm. We will first train our model with lots of images of cats and dogs so that it can learn about different features of cats and dogs, and then we test it with this strange creature. So as support vector creates a decision boundary between these two data (cat and dog) and choose extreme cases (support vectors), it will see the extreme case of cat and dog. On the basis of the support vectors, it will classify it as a cat.

Consider the below diagram:

Types of SVM

SVM can be of two types:

- **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.
- **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

Hyperplane and Support Vectors in the SVM algorithm:

Hyperplane: There can be multiple lines/decision boundaries to segregate the classes in n -dimensional space, but we need to find out the best decision

boundary that helps to classify the data points. This best boundary is known as the hyperplane of SVM.

The dimensions of the hyperplane depend on the features present in the dataset, which means if there are 2 features (as shown in image), then the hyperplane will be a straight line.

And if there are 3 features, then the hyperplane will be a 2-dimension plane.

We always create a hyperplane that has a maximum margin, which means the maximum distance between the data points.

Support Vectors:

The data points or vectors that are the closest to the hyperplane and which affect the position of the hyperplane are termed as Support Vector. Since these vectors support the hyperplane, hence called a Support vector.

Program:

```
#Import required basic libraries
import numpy as np #linear algebra
import pandas as pd #data processing, CSV file I/O (e.g. pd.read_csv) import os

# Read dataset into dataframe
data = pd.read_csv('/kaggle/input/human-activity-recognition-with-smartphones/train.csv')
data.describe()
data

data.info()
data.columns
data.Activity.value_counts()
data.shape
data['activity_code'] = data.Activity.astype('category').cat.codes data.shape
data.activity_code

data1 = data.drop('Activity', axis=1)
data1.shape

x_col = data1.columns.to_list()
x_col.pop(-1)
x_data = data1[x_col]
y_col = 'activity_code'
from sklearn.model_selection import train_test_split
train_x, test_x, train_y, test_y = train_test_split(data1[x_col], data1[y_col].values, test_size=0.1)
train_x.shape, test_x.shape, train_y.shape, test_y.shape
test_y
```

```

#TrytobuildLogisticregression
fromsklearn.linear_modelimportLogisticRegression
from sklearn.metrics import accuracy_score
lg=LogisticRegression(#max_iter=800,solver='sag')
lg.fit(train_x,train_y)
train_y_pred=lg.predict(train_x)
test_y_pred=lg.predict(test_x)
#
print("TrainingAccuracy",accuracy_score(train_y,train_y_pred))
print("Testing Accuracy", accuracy_score(test_y,test_y_pred))

fromsklearn.metricsimportconfusion_matrix
confusion_matrix(train_y,train_y_pred)

confusion_matrix(test_y,test_y_pred)

```

SupportVectorClassifier:

```

SVC(*, C=1.0, kernel="rbf", degree=3, gamma="scale", coef0=0.0,
shrinking=True, probability=False, tol=1e-3, cache_size=200,
class_weight=None, verbose=False, max_iter=-1,
decision_function_shape="ovr", break_ties=False, random_state=None)

```

Parameters

C: float, default=1.0 Regularization parameter. The strength of the regularization is inversely proportional to C. Must be strictly positive. The penalty is a squared l2 penalty.

kernel : {'linear', 'poly', 'rbf', 'sigmoid', 'precomputed'} or callable, default='rbf' Specifies the kernel type to be used in the algorithm.If none isgiven, 'rbf' will be used. If a callableisgiven itisused to pre-computethekernelmatrixfromdatamatrices;thatmatrixshouldbeanarrayof shape (n_samples, n_samples).

degree: int, default=3 Degree of the polynomial kernel function ('poly'). Ignored by all other kernels.

gamma: {'scale','auto'} orfloat,default='scale'Kernelcoefficientfor'rbf','poly'and'sigmoid'.

coef0: float, default=0.0 Independent term in kernel function. It is only significant in 'poly' and 'sigmoid'.

shrinking: bool,default=TrueWhethertouse theshrinkingheuristic.Seethe:ref:UserGuide <shrinking_svm>.

probability: bool, default=False Whether to enable probability estimates. This must be enabled priortocalling fit,willslowdownthatmethodasitinternallyuses5-foldcross-validation,

and `predict_proba` may be inconsistent with `predict`. Read more in the :ref:`User Guide`

`<scores_probabilities>`.

tol: float, default=1e-3 Tolerance for stopping criterion.

cache_size: float, default=200 Specify the size of the kernel cache (in MB).

class_weight: dict or 'balanced', default=None Set the parameter C of class i to `class_weight[i]*C` for SVC. If not given, all classes are supposed to have weight one. The "balanced" mode uses the values of `y` to automatically adjust weights inversely proportional to class frequencies in the input data as `n_samples / (n_classes * np.bincount(y))`.

verbose: bool, default=False Enable verbose output. Note that this setting takes advantage of a per-process runtime setting in libsvm that, if enabled, may not work properly in a multithreaded context.

max_iter: int, default=-1 Hard limit on iterations within solver, or -1 for no limit.

decision_function_shape: {'ovo', 'ovr'}, default='ovr' Whether to return a one-vs-rest ('ovr') decision function of shape (n_samples, n_classes) as all other classifiers, or the original one-vs-one ('ovo') decision function of libsvm which has shape (n_samples, n_classes*(n_classes-1)/

2). However, one-vs-one ('ovo') is always used as multi-class strategy. The parameter is ignored for binary classification.

break_ties: bool, default=False If true, decision_function_shape='ovr', and number of classes > 2, term: predict will break ties according to the confidence values of :term: decision_function; otherwise the first class among the tied classes is returned. Please note that breaking ties comes at a relatively high computational cost compared to a simple predict.

random_state: int, RandomState instance or None, default=None Controls the pseudo random number generation for shuffling the data for probability estimates. Ignored when probability is False. Pass an int for reproducible output across multiple function calls. See :term: Glossary <random_state>.

```
# Build SVM model using svm.SVC with default parameters. Default kernel is rbf
from sklearn.svm import SVC
sv = SVC() # default kernel is 'rbf'
sv.fit(train_x, train_y)
train_y_pred = sv.predict(train_x)
test_y_pred = sv.predict(test_x)
#
print("Training Accuracy", accuracy_score(train_y, train_y_pred))
print("Testing Accuracy", accuracy_score(test_y, test_y_pred))
```

```
# Build SVM model using svm.SVC with Linear Kernel
from sklearn.svm import SVC
sv = SVC(kernel='linear')
sv.fit(train_x, train_y)
train_y_pred = sv.predict(train_x)
test_y_pred = sv.predict(test_x)
#
print("Training Accuracy", accuracy_score(train_y, train_y_pred))
print("Testing Accuracy", accuracy_score(test_y, test_y_pred))
```

```
# Build SVM model using svm.SVC with Polynomial kernel
from sklearn.svm import SVC
sv = SVC(kernel='poly')
sv.fit(train_x, train_y)
```

```
train_y_pred=sv.predict(train_x)
test_y_pred= sv.predict(test_x) #
print("TrainingAccuracy",accuracy_score(train_y,train_y_pred)) print("Testing
Accuracy", accuracy_score(test_y,test_y_pred))
```

```
#BuildSVMmodelusingsvm.SVCwithSigmoidkernel from
sklearn.svm import SVC sv=SVC(kernel='sigmoid')
sv.fit(train_x,train_y)
train_y_pred=sv.predict(train_x)
test_y_pred= sv.predict(test_x)
```

```
#
print("TrainingAccuracy",accuracy_score(train_y,train_y_pred)) print("Testing
Accuracy", accuracy_score(test_y,test_y_pred))
# BuildSVMmodelusingsvm.SVCwithSigmoidkernelandgammaasauto
from sklearn.svm import SVC
sv=SVC(kernel='sigmoid',gamma='auto')
sv.fit(train_x,train_y)
train_y_pred=sv.predict(train_x)
test_y_pred= sv.predict(test_x)
#
print("TrainingAccuracy",accuracy_score(train_y,train_y_pred)) print("Testing
Accuracy", accuracy_score(test_y,test_y_pred))
RESULT:
```

TrainingAccuracyofSVCwithoutKernelis0.9552599758162031 Testing
Accuracy of RFC without Kernel is0.9497282608695652

TrainingAccuracyofSVCwithlinearKernelis0.9936517533252721
Testing Accuracy of SCV with linear Kernel
is0.9836956521739131

TrainingAccuracyofSVCwithpolyKernel0.9578295042321644
Testing Accuracy of SVC with Poly Kernel 0.9565217391304348

TrainingAccuracyofSVCwithsigmoidKernel 0.3636638452237001
Testing Accuracy of SVC with sigmoid Kernel
0.3451086956521739

TrainingAccuracyofSVCwithsigmoidKernel andgammaautois
0.3659310761789601
TestingAccuracyofSVC withsigmoidKernel andgammaautois
0.3464673913043

10.(b) Implement Classification on a sample data set using SVM?

Source Code:

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# Generate a sample dataset
X, y = make_classification(
    n_samples=200, # Number of samples
    n_features=2, # Number of features
    n_classes=2, # Number of classes
    n_informative=2,
    n_redundant=0,
    random_state=42
)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Create an SVM model
svm_model = SVC(kernel='linear', random_state=42)

# Train the model
svm_model.fit(X_train, y_train)

# Make predictions
y_pred = svm_model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

# Print results
print(f'Accuracy: {accuracy * 100:.2f}%')
print("\nClassification Report:\n")
print(report)

# Visualize the decision boundary (optional)
```

```
import matplotlib.pyplot as plt
```

```
def plot_decision_boundary(X, y, model):  
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1  
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1  
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01),  
                          np.arange(y_min, y_max, 0.01))  
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])  
    Z = Z.reshape(xx.shape)  
    plt.contourf(xx, yy, Z, alpha=0.8, cmap=plt.cm.coolwarm)  
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap=plt.cm.coolwarm)  
    plt.title("SVM Decision Boundary")  
    plt.xlabel("Feature 1")  
    plt.ylabel("Feature 2")  
    plt.show()  
  
# Plot decision boundary  
plot_decision_boundary(X_test, y_test, svm_model)
```

Sample Output

You will see an output like this:

markdown

CopyEdit

Accuracy: 95.00%

Classification Report:

	precision	recall	f1-score
support			
0	0.95	0.95	0.95
31			
1	0.94	0.94	0.94
29			

accuracy			0.95	60
macroavg	0.95	0.95	0.95	60
weightedavg	0.95	0.95	0.95	60